Arrays

This lesson will look at how to work with arrays in JavaScript. We will see how we can create arrays , access elements of an array and various other functionalities with arrays.

Introduction

This is a quick revision of Javascript arrays for the sake of completion. If you understand Javascript arrays already, it will be ***quite boring*** *for you.*

In most programming languages, an array is a linear collection of elements stored in a sequential order. We can iterate over arrays using array indices. Array indices typically start from 0 and go upto Array Length - 1. Such arrays are known as zero-based arrays. Javascript arrays are zero based arrays as well.

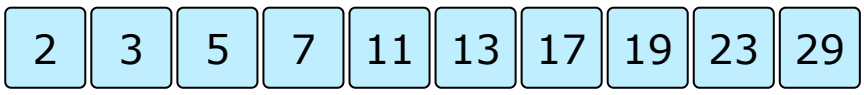
However, one unique thing about Javascript arrays is that they are actually objects and indices are just property names in that object. Indices are usually numbers and so Javascript arrays convert these numerical indices to string keys and then assign them as object properties. Therefore, if you access array[0], you are actually accessing property "0" of object array.

Here's how we can create an array in javascript.

```
var topics = [];
```

Initializing the array

You can also initialize the array by passing in the values when declaring the array. e.g. let's create an array of the first te.n prime numbers.



First 10 prime numbers

```
var primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29];
```

Adding elements to an existing array

Once you have created the array, you can add more elements to the array using **push** method.

```
var topics = [];
topics.push('Arrays');
topics.push('Stacks');
topics.push('Queues');
```

Length of the array

You can get the length of the array using the '**length**' property.

```
var topics = [];
console.log('[' + topics + '] has length = ' +
            topics.length);

topics.push('Arrays');
topics.push('Stacks');
topics.push('Queues');

console.log('[' + topics + '] has length = ' +
            topics.length);
```

Accessing Array Elements

Array elements are accessed using the indices ranging from 0 to length -1. Here's some code.

```
var topics = [];

topics.push('Arrays');
topics.push('Stacks');
topics.push('Queues');

for (var i = 0; i < topics.length; i++) {
  console.log(topics[i]);
```

```
}
```

The 'new' Array Syntax

Arrays can also be created with the new keyword in JavaScript as array is an object. In most cases, this way of creating arrays in inefficient and should be avoided. However, for learning, here's the syntax.

```
var topics = new Array('Arrays', 'Stacks', 'Quotes');
```

Let's have a quick quiz

Quiz 1

Q

Array ‘tutorial’ is defined as

```
var tutorial=['Arrays', 'Stacks', 'Queues'];
```

What’s the correct syntax to replace ‘Queues’ with ‘Linked Lists’?

Check Answers

Removing elements from an array

Elements of an existing array can be removed or permanently deleted by using the splice method. Splice method requires two arguments at the minimum, the index of first element to delete and the number of elements to delete starting from the given index.

```
var tutorials = ['Arrays', 'Stacks', 'Queues'];
console.log('Before Splice: ' + tutorials);
tutorials.splice(1,1);
console.log('After Splice: ' + tutorials);
```

Stacks

Stack is a Last In, First Out data structure useful is solving many problems in computer science.

Introduction

A stack is a list like data structure which allows three basic operations.

1. Insert into the stack. This is known as ***Push***.
2. Extract the most recently added. This is called ***Pop***.
3. Look at the most recently added element without removing (popping) it. This is called ***Peek***.

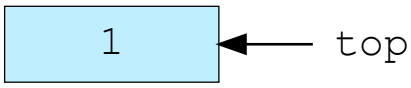
Using the above terminology, we can say that we push elements into the stack and we pop elements from the stack. Stack is known as a ***Last-In First-Out (LIFO)*** data structure.

To support operations in stack, we always keep track of the top most element in the stack. Here's how tracking the ***top*** element in the stack helps us implement the above operations.

1. When we push an element, we update the top to start pointing to the newly added element.
2. When we pop an element, we update the top to point to the next most-recently-added element in the stack.
3. When we peek, we just return the data pointed by the top.

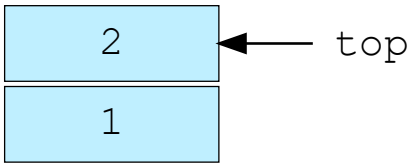
Here's a quick animation where we push and pop elements in the stack.

Push (1)



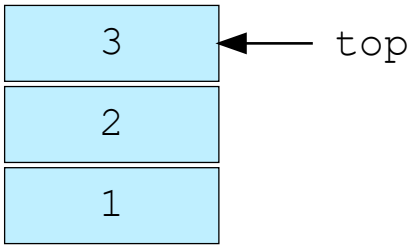
Let's insert the first element in the stack.
Pushing 1 on the stack

Push (2)



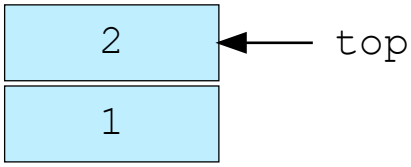
Push 2 on the stack. Look how Top starts pointing to the newly added element.

Push (3)



Element of value 3 is pushed onto the stack.
It now becomes the top most element on the stack

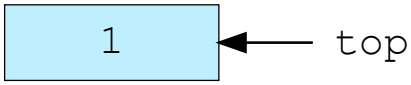
Pop() returns 3



Pop an element from stack. It returns 3.
Top now points to 2.

4 of 5

Pop() returns 2



Pop returns 2. Top starts pointing to 1

5 of 5

Defining the stack

The below block of code shows how a stack would be defined in JavaScript

```
function Stack() {
  this._top = -1;
  this._values = [];
}
```

The Stack function would contain 2 parts.

- The index of the most recent element tracked by *_top*. It is initialized by -1 to indicate that stack contains no elements.

to indicate that stack contains no elements.

- Array of values inserted into the stack tracked by *_values*. It is initialized by an empty array.

Pushing elements onto the stack

Let's implement the push operation. The below block of code defines how elements can be pushed or added to the stack.

```
Stack.prototype.push = function(data) {
  this._top++;
  this._values[this._top] = data;
};
```

We just increment the top to get the index for new element and then insert the data. Top is automatically pointing to the top-most element in the stack

Popping elements from the stack

Let's look at how can we implement pop operation on our stack. First, we check that stack has any element. If stack is empty, we return null. We then note down the element at the top, then reduce the top and reduce the size of the array. As the last step we return the element that was at the top.

```
Stack.prototype.pop = function() {
  if (this._top < 0) {
    return null;
  }

  var topElement = this._values[this._top];
  this._top--;

  //We could have used array pop method here to remove
  //the last array element but we haven't used it to
  //avoid confusion with Stack's pop method
  this._values.length--;

  return topElement;
};
```

Peeking at the top element

Implementing peek is easy. We just return the element pointed by the top.

Here's the code which is self explanatory.

```
Stack.prototype.peek = function() {
  if (this._top < 0) {
    return null;
  }

  return this._values[this._top];
};
```

Stack in action

Let's run the functions that we just discussed and see how the stack works. We'll push a few elements and then pop them.

```
var stack = new Stack();

for (var i = 100; i <= 300; i+=100) {
  console.log('Pushing on stack: ' + i);
  stack.push(i);
}

console.log('Top of the stack using Peek: ' + stack.peek());
console.log('Pop from stack. Popped element = ' + stack.pop());
console.log('Pop from stack. Popped element = ' + stack.pop());
```

Let's have a quick quiz

Stack - Basics

1 What are the two most common operations on a stack?

2

Both return the top value of stack. Is Peek Operation different from Pop Operation?

Check Answers

Exercise 1

As a first exercise, let's implement a method size that returns the number of elements in the stack.

> Some tests are failing right now. Implement 'size' to fix them.

JavaScript

HTML

CSS (SCSS)

```
Stack.prototype.size = function() {
  ///// TODO. Implement this /////
  return undefined;
}

// This is the function that runs test cases.
runEvaluation();
```

Console

Clear

```
*** There is some bug lurking there. See failed test cases ***
Here are the tests that ran:
Test case FAILED for stack.size(). Result: undefined. Expected: 0
```

```
Test case FAILED for stack.size(). Result: undefined. Expected: 2
```

```
Test case FAILED for stack.size(). Result: undefined. Expected: 1
```

Exercise 2

As a second exercise, let's implement isEmpty method. It returns true when stack is empty and returns false when the stack has one or more elements.

> Some tests are failing right now. Implement 'isEmpty' to fix them.

JavaScript

HTML

CSS (SCSS)

```
Stack.prototype.isEmpty = function() {
  ///// TODO. Implement this /////
  // Return true when empty //
  // Return false when not empty
  return undefined;
}

// This is the function that runs test cases.
runEvaluation();
```

Console

Clear

```
*** There is some bug lurking there. See failed test cases ***
```

```
Here are the tests that ran:
```

```
Test case FAILED for stack.isEmpty(). Result: undefined. Expected: true
```

```
Test case FAILED for stack.isEmpty(). Result: undefined. Expected: false
```

```
Test case FAILED for stack.isEmpty(). Result: undefined. Expected: true
```

Does Javascript has a built-in Stack?

Yes. In fact, the Array in Javascript acts as a stack and already has the appropriate methods to use it like stack. Look at the below code.

```
var stack = [];
console.log('Push 100 in stack');
stack.push(100);
console.log('Popped from Stack: ' + stack.pop());
```

Summary

- Stack is a LIFO data structure
- Stack has 3 basic operations: Push, Pop and Peek.
- Javascript Array is actually a stack and supports Push and Pop.

Queues

Queue is a FIFO data structure. In this part, we look at Queues and how can they be implemented in Javascript

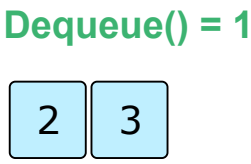
Introduction

A queue is a linear collection where items are inserted at the end and are removed from the front. Hence queue is a ***FIFO (first-in first-out)*** data structure. It supports three basic operations: Insert and Remove.

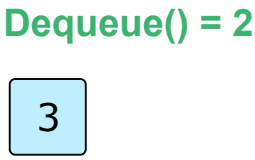
1. Inserting into the queue is called ***Enqueue***.
2. Extracting from the queue is called ***Dequeue***.
3. Look at the top element in the queue (the one that will be removed on Dequeue). It is called ***Peek***.

Queue is an abstraction of the behavior that is very common in our daily lives e.g. drive through of a Coffee Shop. It helps process requests or tasks in the order that they arrive. Here's a quick animation on how queue works.

3 of 5



4 of 5



5 of 5

> Queue is often compared and studied along with Stack which is a LIFO data structure.

Queue Implementation

We'll implement the Queue data structure using the following object.

```
function Queue() {
  this._head = 0;
  this._data = [];
}
```

We are going to store the queue items in an array.

- *data* contains the items in the queue. At every enqueue, we push the new item into this array.
- *head* is the index of the top or head of the queue. At every dequeue, we return the element pointed by the head and then increment the head to point to the next item.

Let's look at the code for Enqueue and Dequeue below.

Enqueue elements in the Queue

Here's how the Enqueue function would look.

```
Queue.prototype.enqueue = function(data) {
  this._data.push(data);
};
```

Dequeue items from the queue

Here's how a (*rather simplistic*) Dequeue implementation would look. If you have figured out an issue with this method along the lines of excessive memory usage stay with us it will be discussed later on.

```
Queue.prototype.dequeue = function() {
  if (this._head < 0 ||
      this._head >= this._data.length) {
    return null;
  }

  var dequeuedItem = this._data[this._head];
  this._head++;

  return dequeuedItem;
};
```

Peek at the queue

Here's how peek function would look. It's very similar to the Dequeue operation except the head doesn't move to the next item.

```
Queue.prototype.peek = function() {
  if (this._head < 0 ||
      this._head >= this._data.length) {
    return null;
  }

  return this._data[this._head];
};
```

Let's look at the complete implementation of queue.

> Run the following code to see Queue in action

JavaScript

HTML

CSS (SCSS)

```
var queue = new Queue();
console.log("Enqueue 100");
queue.enqueue(100);

console.log("Enqueue 200");
queue.enqueue(200);

console.log('Dequeue: ' + queue.dequeue());
console.log('Dequeue: ' + queue.dequeue());
console.log('Dequeue: ' + queue.dequeue());
```

Console

Clear

```
Enqueue 100
Enqueue 200
Dequeue: 100
Dequeue: 200
Dequeue: null
```

Queues Basics

1 Is Queue a LIFO data structure?

2 Which operation decreases the length of the queue?

Check Answers

Exercise 1

As the first exercise, let's implement the *size* function. It returns the size of the queue.

> Some test cases are failing. Fix the size method and click Run button.

JavaScript

HTML

CSS (SCSS)

```
Queue.prototype.size = function() {
  // TODO:
  // IMPLEMENT THIS
  return undefined;
}

// This is the function that runs test cases.
runEvaluation();
```

Console

Clear

```
*** There is some bug lurking there. See failed test cases ***
Here are the tests that ran:
Test case FAILED for queue.size(). Result: undefined. Expected: 0
Test case FAILED for queue.size(). Result: undefined. Expected: 1
```

```
Test case FAILED for queue.size(). Result: undefined. Expected: 0
```

Exercise 2

As a next exercise, let's implement the *isEmpty* function. It returns true when queue has no items and returns false otherwise.

> Some test cases are failing. Fix the isEmpty method so that they can pass.

JavaScript

HTML

CSS (SCSS)

```
Queue.prototype.isEmpty = function() {
  // TODO:
  // IMPLEMENT THIS
  return undefined;
}

// This is the function that runs test cases.
runEvaluation();
```

Console

Clear

```
*** There is some bug lurking there. See failed test cases ***
Here are the tests that ran:
Test case FAILED for queue.size(). Result: undefined. Expected: true
Test case FAILED for queue.size(). Result: undefined. Expected: false
Test case FAILED for queue.size(). Result: undefined. Expected: true
```

Our dequeue implementation leaks memory

For learning purposes, we used a simpler version of Dequeue that leaks memory. Can you figure out how?

When we dequeue, we never release the items from the array. With every dequeue, we move the head forward but the array still holds the reference to the item. It means, all the elements in the array at indexes less than the head are garbage.

To fix this, we cleanup the array. We don't have to slow down each dequeue as deleting from the front of the array is a slow operation. We only do it after a certain threshold. e.g. we can say that we only cleaup the array when the garbage is 10 items or 100 items. Let's look at the code for a dequeue method where we cleanup periodically.

```
Queue.prototype.dequeue = function() {
  if (this._head < 0 ||
      this._head >= this._data.length) {
    return null;
  }

  var dequeuedItem = this._data[this._head];
  this._head++;

  if (this._head === 100) {
    // We have 100 items in garbage
    // Remove items at indexes 0 to 99.
    this._data.splice(0, 100);

    // Reset the head
    this._head = 0;
  }

  return dequeuedItem;
};
```

Does Javascript has built-in support for Queues

It does. Just like stacks, queues can be implemented using Javascript's array. Here's how.

Enqueue can be implemented by using Array.push

Dequeue can be implemented by using Array.shift which removes the first element from the array.

However, you can see that calling Shift at each dequeue is slow as it mutates the array. Hence, if you are writing performant code, the array we implemented is better as it has periodic/lazy cleanup instead of an aggressive cleanup.

Summary

- A queue is a first-in-first-out (FIFO) data structure
- It supports Enqueue for insertion and Dequeue for extraction of items.
- Peek operation return the top most item in the queue.
- Javascript array supports Enqueue (Array.push) and Dequeue (Array.shift)
- Implementing dequeue using Array.shift can be slow.
- Dequeue operation should do periodic cleanup to save memory.

Sets

This chapter will look at how to work with Sets in JavaScript. We will see at how we can create Sets , access elements of a Set and various other functionalities on Sets.

Introduction

A set is an unordered collection of related members in which no member occurs more than once. A set that doesnt contain any element is often called null set. Below is a typical representation of set:

The most common set operations are given below

Union - This is wherein a new set is constructed by combining the members of one set with the members of another set.

Intersection - This is wherein a new set is constructed by adding all the members of one set that also exist in a second set.

Difference - This is wherein a new set is constructed by adding all the members of one set except those that also exist in a second set.

Let's now look at how we can manage and work with sets using JavaScript.

Set Representation in JavaScript

The below code snippet is the most typical representation of a set in javacsript

```
function Set() {
  this.set = {};
  this.size = 0;
}
```

The set is represented as an Object and the values of the set can be inserted as properties. The size is maintained separately using a size variable. There are other ways of implementing sets in Javascript. We have picked Object for it's simiplicity.

Adding items to the set

Now let's look at how we can add elements into our set. The below snippet of code showcases how we can add values to our set data structure.

```
Set.prototype.add = function(data) {
  if (!(this.set.hasOwnProperty(data))) {
    this.set[data] = 'true';
    this.size++;
  }
}
```

The first check is done to see if the data actually exists in our set. This is done by using the hasOwnProperty method which is available for Javascript objects. If not then the data value is added to the set by setting the property of the object to the data which is passed to the add function. The associated value is set to true , just to indicate that the property is part of the set. Technically in this implementation of set the value can be set to anything.

Removing an item from the set

Below is the code snippet that can be used for removing an item from the set

```
Set.prototype.remove = function(data) {
  if (this.set.hasOwnProperty(data)) {
    delete this.set[data];
    this.size--;
  }
}
```

First we ensure that the data value to be removed is available in the set. This is done by using the hasOwnProperty function of JavaScript objects. If the property does exist , then it is removed from the set data structure. At the

same time, the size value is decremented to indicate that an element has been removed from the set.

> Run the code below to see add in action

JavaScript

HTML

CSS (SCSS)

```
var myset = new Set();
myset.add(1);
myset.add(2);
myset.add(3);
myset.print();
console.log('removing 2');
myset.remove(2);
myset.print();
```

Console

Clear

```
{1, 2, 3}
removing 2
{1, 3}
```

Membership in the set

One of the key functionalities of sets is the determination of whether a value actually belongs to a set. The following code snippet shows how this can be accomplished.

```
Set.prototype.member = function(data) {
  if (this.set.hasOwnProperty(data)) {
    return true;
  } else {
    return false;
  }
}
```

The above code uses a partial implementation of what is used for remove

functionality of sets.

Quiz 1

Q

An ideal representation of a set in javascript is represented as

Check Answers

Exercise 1

As a first exercise, let's implement a method size that returns the number of items in set.

> Some tests are failing right now. Implement 'size' to fix them.

JavaScript

HTML

CSS (SCSS)

```
Set.prototype.sizeOfSet = function() {
  /////// TODO ///////
  ////// IMPLEMENT THIS /////
  ////// RUN AFTER IMPLEMENTING THIS /////
  ////// TESTS SHOULD PASS /////
  return undefined;
}
```

```
// This is the function that runs test cases.
runEvaluation();
```

Console

Clear

```
*** There is some bug lurking there. See failed test cases ***
Here are the tests that ran:
Test case FAILED for Empty Set Size. Result: undefined. Expected: 0
Test case FAILED for Set Size. Result: undefined. Expected: 1
Test case FAILED for Set Size. Result: undefined. Expected: 2
```

Union of two sets

Now let's look at how we can carry out the union of two sets in Javascript. The union of two sets returns a set containing all elements in either this or the other set. Let's look at the code snippet which can be used to accomplish this.

```
Set.prototype.union = function(secondset) {
  var unionset = new Set();
  for (var key in this.set) {
    if (this.set.hasOwnProperty(key)) {
      unionset.add(key);
    }
  }
  for (var key in secondset.set) {
    if (!unionset.set.hasOwnProperty(key)) {
      unionset.add(key);
    }
  }
  return unionset;
}
```

Intersection of two sets

The intersection of two sets returns a set containing all elements that exist in both sets. Here goes the implementation for intersection for our set data structure

```
Set.prototype.intersect = function(secondset) {
  var inter = new Set();
  for (var key in this.set) {

    if (secondset.set.hasOwnProperty(key)) {
      inter.add(key);
    }
  }
  return inter;
}
```

Sidebar on ES6

ES6 has a built-in Set object. It is implemented now in some browsers and in near future will get full support. We recommend that instead of implementing your own Set structure leverage the one thats provided by ES6. A key advantage of this Set Object is that it doesn't coerce all keys to a string like the Object does so you can have both 6 and "6" as separate keys.

Exercise 2

Let us implement a set different (A - B) method. Fix the following function to return all the elements that exist in the current set but not in the *otherset* passed as parameter.

> Run the following code. Implement difference method so that the tests pass.

JavaScript

HTML

CSS (SCSS)

```
Set.prototype.difference = function(secondset) {
  //TODO IMPLEMENT THIS
}


// This is the function that runs test cases.
runEvaluation();
```

Console

Clear

```
*** There is some bug lurking there. See failed test cases ***
```

```
Here are the tests that ran:
```

```
Test case FAILED for Difference of two empty sets. Result: undefined. Expected: 0
```

```
Test case FAILED for Difference of two sets. Result: undefined. Expected: 5
```

Summary

- A set is a data structure which has the following key properties:
  - Elements are unordered
  - No Duplicates
- The add and remove functions are used to add and remove items from the set.
- The union of two sets returns a set containing all elements in either this or the other set
- The intersection of two sets returns a set containing all elements in both this and the other set.

Dictionary

Dictionaries are key-value pairs used for easy lookups. They are also called associative arrays as each entry into the dictionary is a key with an associated value. In this tutorial, we look at how to implement dictionaries in Javascript, how to handle duplicate keys and how to implement lookups and deletes.

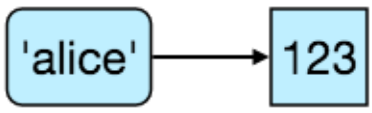
Introduction

Dictionary is a data structure that consists of key-value pairs. It's a data structure where a key is associated with a value. Hence, they are also called ***associative arrays***. Dictionaries provides lookups on its keys.

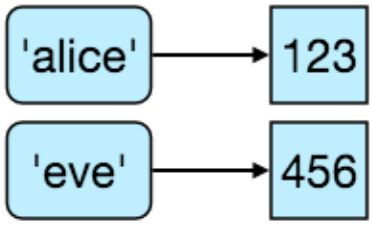
Let's go through a quick visualization

Let's save phone's of Alice, Eve and Bob in a dictionaryof

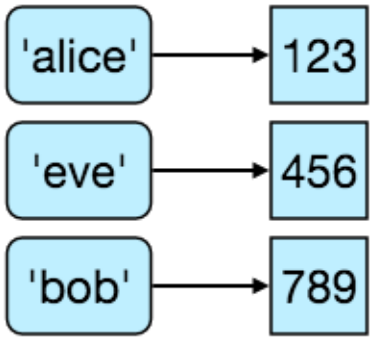
1 of 4



Alice's phone is 123



Eve's phone is 456



Bob's phone is 789

Duplicates in Dictionary

We are not going to handle duplicates in our dictionary. Most implementations of associative arrays don't support duplicates as it is mostly thought of as a 1:1 relationship between a key and a value instead of a 1:N relationship. However, if you are in a situation where you need to associate duplicate values to a key, it's not that difficult to implement either.

Key Types

For simplicity, we are going only to accept strings or integers as keys. Why not other types of keys? We can do it, but we have to then define a standard way

other types of keys. We can do it, but we have to then define a standard way

to compare and equate objects. For now, we'll keep it simple as Javascript already provides a way to compare integers and strings.

We are also going to consider an integer and its string representation as the same key e.g. 45 and "45" will refer to the same key.

Why don't we use Javascript Object?

You are right. Javascript object is already an associative array. Why don't we use it? We will use it internally but with our implementation, we can provide cleaner methods around our Dictionary e.g. we can provide a length method. We can also provide methods to enumerate keys and values.

Let's start implementing the dictionary.

Implementing Dictionary in Javascript

We'll use Javascript's object for our key/value pair storage.

```
function Dictionary() {
  this._data = {};
  this._length = 0;
}
```

Adding a element to the dictionary

First we'll ensure that the key is a string or an integer. In addition, if someone tries to add a key that already exists, we will throw an exception (this is again a matter of preference and the other option is to overwrite the value). We use ***Object.hasOwnProperty*** method to check for duplicates.

```
Dictionary.prototype.add = function(key, value) {
  var keyType = typeof(key);

  if (keyType !== 'string'  && keyType !== 'number') {
    throw 'Key type can only be string or a number';
  }

  if (this._data.hasOwnProperty(key)) {
    throw 'Duplicate keys are not supported';
  }
```

```
  this._data[key] = value;

  this._length++;
}
```

Looking up a value in the dictionary

Next let's look how to implement the lookup method. It's pretty straightforward.

```
Dictionary.prototype.find = function(key) {
  if (key === null) {
    return undefined;
  }

  var keyType = typeof(key);
  if (keyType !== 'string'  && keyType !== 'number') {
    return undefined;
  }

  if (this._data.hasOwnProperty(key)) {
    return this._data[key];
  }

  return undefined;
}
```

Removing an element from the Dictionary

We'll use ***delete*** function to remove both the key and its associated value from our data object. Here's the code for that.

```
Dictionary.prototype.remove = function(key) {
  if (this._data.hasOwnProperty(key)) {
        delete this._data[key];
        this._length--;
  }
}
```

Dictionary in action

> Run the following code to see dictionary in action

JavaScript

HTML

CSS (SCSS)

```
var dict = new Dictionary();
dict.add("alice", 123);
dict.add("eve", 456);
dict.add("bob", 789);

console.log('dict.find(\'alice\') = '
            + dict.find('alice'));
console.log('dict.find(\'foo\') = '
            + dict.find('foo'));
```

Console

Clear

```
dict.find('alice') = 123
```

```
dict.find('foo') = undefined
```

Exercise 1

> *Implement the size method to fix the code below*

JavaScript

HTML

CSS (SCSS)

```
Dictionary.prototype.size = function() {
  // Implement this function
}

// Don't edit.
// This runs tests
runEvaluation();
```

Console

Clear

```
*** There is some bug lurking there. See failed test cases ***
```

```
Here are the tests that ran:
```

```
Test case FAILED for dict.size(). Result: undefined. Expected: 0
Test case FAILED for dict.size(). Result: undefined. Expected: 1
Test case FAILED for dict.size(). Result: undefined. Expected: 1
```

Summary

Dictionary is a collection of key/value pairs.

It's also called an associative array as every key is associated with a value.

Some dictionary implementations support multiple values associated with a single key.

Some dictionary implementations allow overwriting the value associated with the key. Others don't allow this in which case you would have to first delete the key and insert it again with the new value.

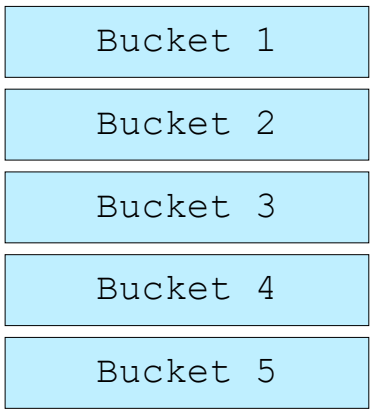
Hash Table

What is a Hash Table?

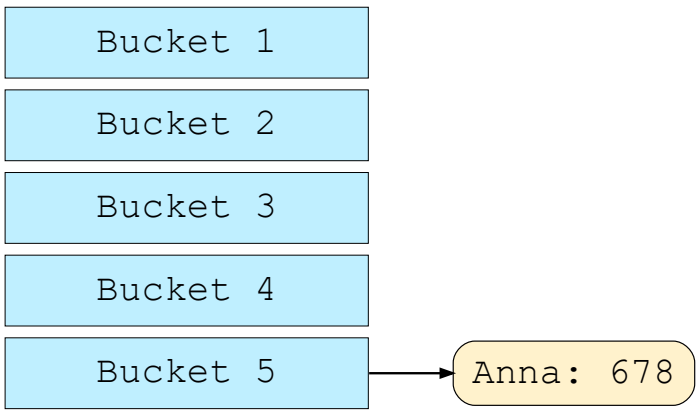
A Hash table is a data structure that is used to implement an associative array of a key and value. But we said the same thing about the Dictionary. How is different from a dictionary?

A dictionary is a 1:1 map of a key/value pair. Hashtable is a more efficient way of storing key/value pairs. It is designed around an array where each array cell is called a ***bucket***. Each bucket contains a list of key/value pairs. When a key/value pair is inserted into a hashtable, we first compute it's destination bucket. We determine the bucket by applying a special function on the key. This special function is known as the ***hash function***. Every hash function returns a number based on the key. This number is the bucket Id. We then go to the bucket and append this key/value pair to the list of key/value pairs in the bucket.

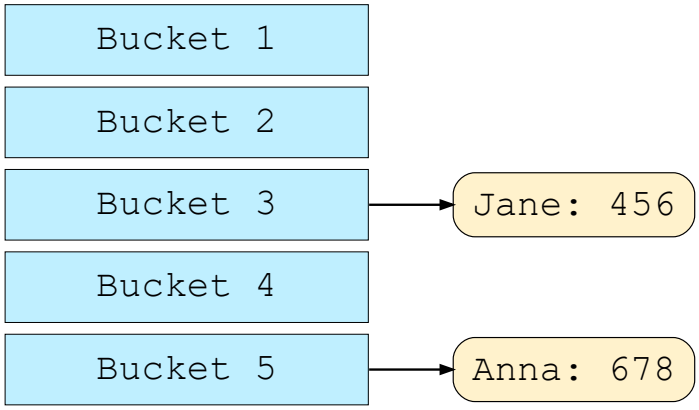
Here's a quick visualization to look at the hash table (Our Hash Table has 5 buckets).



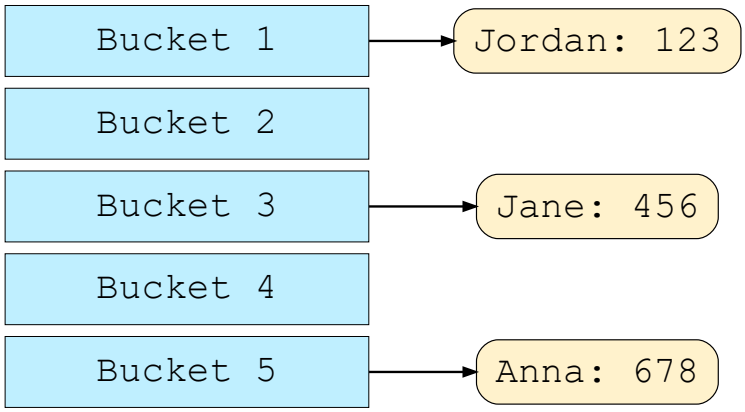
Let's insert phone numbers into a hash table



Insert Anna's Phone: 678.
Key 'Anna' maps to bucket 5.



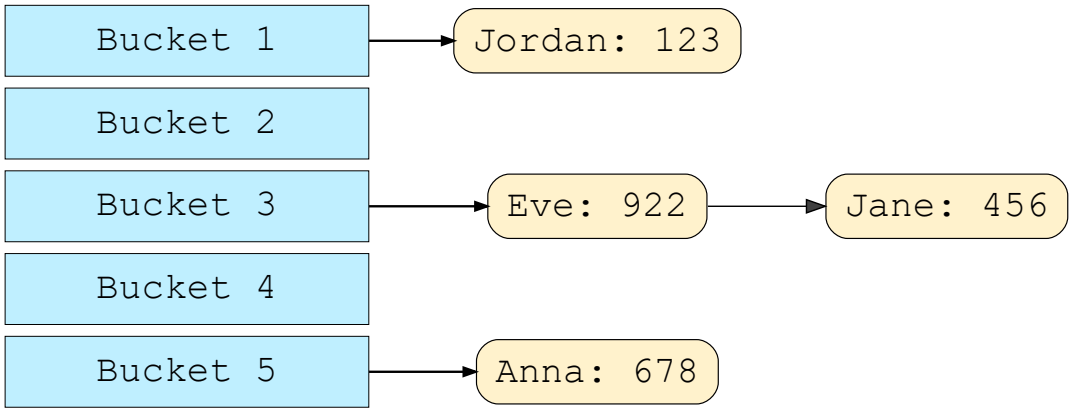
Insert Jane's Phone: 456.
Key 'Jane' maps to bucket 3.



Insert Jordan's Phone: 123.

Key 'Jordan' maps to bucket 1.

4 of 5



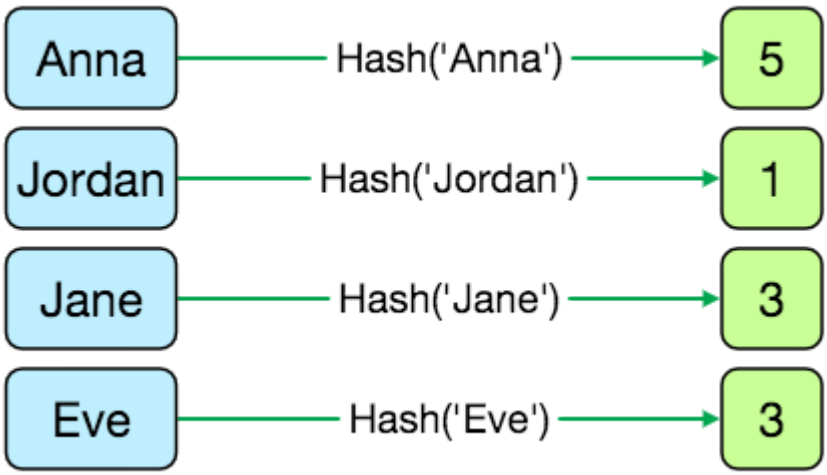
Insert Eve's Phone: 922.
Key 'Eve' maps to bucket 3.

5 of 5

When we have to add a key/value pair to a bucket that already has one or more key value pairs, we say that a ***collision*** has occurred. Whenever a collision occurs, we apply a collision resolution strategy. We'll look into these strategies in a bit. However, lets first focus on the the Hash functions.

Hashing and Hash Function

As we've already seen, Hash tables rely on a hash function to convert keys into a bucket index. The implementations of Hash functions range from simple functions to very sophisticated. The goal of a production quality hash function is to avoid collisions as much as possible. Such hash functions are called uniform hash functions. More formally, a ***uniform hash*** function is a function that maps the expected inputs into available buckets as evenly as possible. If you look at the figure below, it's pretty clear that our hash function is not a uniform one.



Hash function used in our Hash Map Visualization

To keep things simple, we are going to use a very basic hash function but it should give you an idea about how hash functions are implemented. Here's the definition of our Hash Table.

```
function HashTable() {
  this._bucketSize = 23;
  this._buckets = [];
  this._buckets.length = this._bucketSize;
}
```

If you notice, we have set the bucket count to 23. The total number of buckets is usually a prime number (and usually a relatively large one than our pick 23). Now we know that we will have a total of 23 buckets. Let's implement our very basic hash function.

```
HashTable.prototype.computeHash = function(key) {
  var total = 0;
  for (var i = 0; i < data.length; ++i) {
    total += data.charCodeAt(i);
  }

  return total % this._bucketSize;
}
```

We calculate the sum of the character codes of the data and then make sure that the final hash is less than the number of buckets. Based on this hash function, let's implement our Insert function (It's incomplete without handling collisions but we'll get there).

```
HashTable.prototype.put = function(key, value) {
  var hash = this.computeHash(key);

  if (this._buckets[hash] != undefined) {
    throw 'We are not handling collisions yet";
  }

  this._buckets[hash] = value;
}
```

If you look at the above function, it's clear that we can only store one key/value pair in a bucket. In addition, if you notice closely, there's also a bug in the above code. We are not saving the key. We'll run into problems because a hash function can return the same hash for many keys. Hence, in our example above, if you add the phone number of Jane, and the try to get the phone number of Eve, we will return you Jane's number. Hence, the first learning is that we need to store all keys along with their values. We will fix that when we implement the code again but first let's look at collision resolution techniques.

Collision Resolution in Hash Table

There are two basic techniques to handle collisions in a hash table.

1. Separate chaining
2. Open addressing

Let's look at both of them in a little bit more detail.

Separate Chaining

In this method, each bucket contains a key/value pair list itself. In fact, the first visualization we saw (with a collision with Eve and Jane), we handled the collision by separate chaining. Separate chaining is easy to implement and understand. If the hash function is almost uniform and the bucket size is large enough, collisions would be rare and a few buckets with chains of 3 or 4 keys/value pairs is not a big issue. However, if the collisions are frequent, then some buckets would be heavily loaded and lookups within a bucket can be slow (which would significantly reduce the efficacy of our hash table).

Open Addressing

Open Addressing

In this method, instead of having a list within a list, we have a single array of buckets and all key/value pairs are stored in a bucket. The underlying assumption is that the bucket count is large enough to accommodate all key/value pairs. Insert works by looking at the bucket returned by the hash function. If the bucket is occupied, it starts probing for the next bucket using a ***Probing Sequence***.

One most common probing sequence is called ***Linear Probing*** where we start looking for next free bucket by adding a fixed integer to the current bucket. e.g. If hash function returned bucket5 but bucket5 is occupied, then we will start looking at bucket6, bucket7 ... until we find a free bucket. You can see that with this approach we cannot handle key/value pairs larger than the number of buckets. If we have to probe many times before we find a slot, it will not only slow down the insertions, it will also slow down the lookups.

Which collision resolution technique is better?

Both collision resolution techniques that we discussed have pros and cons. You need to pick the technique based on the amount and nature of the data. Typically, hash tables implement a re-hashing when collisions go beyond a certain threshold. However, it's a costly operation. Most hash table implementations would let you specify the number of buckets at its creation. This helps in avoiding collisions and paying the cost of re-hashing.

We'll implement both techniques for our learning here.

Implementing Hash Table with Separate Chaining

Javascript has a built-in object so instead of storing a linked list or array in each bucket, we are going to store an object and use that object to store key/value pairs.

Here's how we will implement our put function using separate chaining

```
HashTable.prototype.put = function(key, value) {
  var keyType = typeof(key);
  if (keyType !== 'string' && keyType !== 'number') {
    throw 'Only string or number keys are supported';
```

```
  }

  var hash = this.computeHash(key);


  if (this._buckets[hash] === undefined) {
    this._buckets[hash] = {};
  }

  var chain = this._buckets[hash];

  if (chain.hasOwnProperty(key)) {
    throw 'Duplicate key is not allowed';
  }

  chain[key] = value;
}
```

and here's the corresponding get function

```
HashTable.prototype.get = function(key) {
  var keyType = typeof(key);
  if (keyType !== 'string' && keyType !== 'number') {
    return undefined;
  }

  var hash = this.computeHash(key);

  if (this._buckets[hash] === undefined) {
    return undefined;
  }

  var chain = this._buckets[hash];

  if (chain.hasOwnProperty(key)) {
    return chain[key];
  }

  return undefined;
}
```

>Run the following program to see Hash Table implemented with separate chaining.

JavaScript

HTML

CSS (SCSS)

```
var contacts = new HashTable();
contacts.put("Anna", 678);
contacts.put("Jordan", 123);
console.log('Anna\'s Phone: ' + contacts.get('Anna'));
console.log('Jordan\'s Phone: ' + contacts.get('Jordan'));
```

```
console.log('Frank\'s Phone: ' + contacts.get('Frank'));
```

Console

Clear

```
Anna's Phone: 678
Jordan's Phone: 123
Frank's Phone: undefined
```

Implement Hash Table with Open Addressing

We'll use linear probing to implement collision resolution. Here's the code for implementing put using linear probing.

```
HashTable.prototype.put = function(key, value) {
  var keyType = typeof(key);
  if (keyType !== 'string' && keyType !== 'number') {
    throw 'Only string or number keys are supported';
  }

  var hash = this.computeHash(key);

  if (this._buckets[hash] === undefined) {
    // Yay No collision found
    this._buckets[hash] = {};
    this._buckets[hash][key] = value;
    return;
  } else if (this._buckets[hash].hasOwnProperty(key)) {
    // Duplicate Key
    throw 'Duplicate Key is not allowed';
  }

  // Collision found.
  // Let's probe for the next available slot
  var bucketId = hash + 1;

  do {
    if (bucketId >= this._bucketSize) {
      // Reached the end.
      // Start from the beginning
      bucketId = 0;
    }

    if (this._buckets[bucketId] === undefined) {
      // Found an empty slot
      this._buckets[bucketId] = {};
      this._buckets[bucketId][key] = value;
      return;
```

```
    }

    bucketId++;
  } while (bucketId != hash);

  // Couldn't find any free slots.
  throw 'Hash Table is full. Rehashing needed';
}
```

and here's the corresponding get function.

```
HashTable.prototype.get = function(key) {
  var keyType = typeof(key);
  if (keyType !== 'string' && keyType !== 'number') {
    return undefined;
  }

  var hash = this.computeHash(key);

  if (this._buckets[hash] === undefined) {
    return undefined;
  } else if (this._buckets[hash].hasOwnProperty(key)) {
    // Key found. Return value
    return this._buckets[hash][key];
  }

  // Possible Collision.
  // Iterate through the table using the
  // probing sequence used by the put function
  var bucketId = hash + 1;

  do {
    if (bucketId >= this._bucketSize) {
      // Reached the end.
      // Start from the beginning
      bucketId = 0;
    }

    if (this._buckets[bucketId] === undefined) {
      // Found an empty slot
      return undefined;
    } else if (this._bucekts[bucketId].hasOwnProperty(key)) {
      // Key found. Return value
      return this._buckets[hash][key];
    }

    bucketId++;
  } while (bucketId != hash);

  // Couldn't find the key and exhausted the
  // whole hash table.
  return undefined;
}
```

>Run the following program to see Hash Table implemented with open addressing (linear probing).

JavaScript

HTML

CSS (SCSS)

```
var contacts = new HashTable();
contacts.put("Anna", 678);
contacts.put("Jordan", 123);
console.log('Anna\'s Phone: ' + contacts.get('Anna'));
console.log('Jordan\'s Phone: ' + contacts.get('Jordan'));
console.log('Frank\'s Phone: ' + contacts.get('Frank'));
```

Console

Clear

```
Anna's Phone: 678
Jordan's Phone: 123
Frank's Phone: undefined
```

Summary

Hash Table is a data structure used to implement key value pairs.

Hash Table consists of a buckets (slots) and a hashing function maps the keys to slots.

Hash function can have collisions. When there is a collision, there are strategies to resolve to collision.

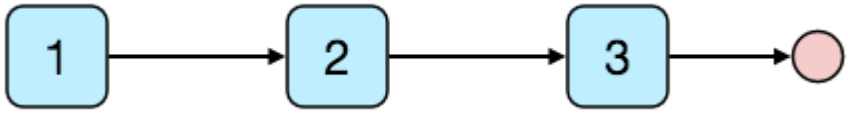
Two most common stragies for conflicit resolution are called separate chaining and open addressing.

Linked Lists

Linked lists are are an ordered set of nodes where each node contains a link to its successor.

Introduction

A linked list is a data structure that holds a sequence of linked nodes. Each node contains data and a reference, where reference points to its successor node in the Linked List.



Linked List with 3 nodes

The above diagram shows a typical example of a singly linked list. It consists of 3 nodes. The first node consists of a value of 1 and contains a pointer to the next node. The next node in the list has a value of 2 and subsequently consists of a reference to the last node in the list and so on. The last node in a linked list contains a value 3 but it doesn't point to anything.

When a reference doesn't point to any valid object, programming languages typically have a "NULL" value to represent the missing value. In Javascript, we use "null" for that purpose - and it's called a *null reference*.

> *Reference is also sometimes called a pointer. This comes from the C/C++ world where variables can point to memory addresses. In Javascript, we cannot point to memory addresses. Instead we have references to objects.*

The first node in the linked list is called ***Head Node***. Head node is our handle to the linked list. We traverse the linked list starting at the head node and then move to the node pointed by the current node. We'll visit the traversal in detail later.

Why another 'List' when we already have an array?

It's a valid question. We already have a built-in type Array that acts as a container similar to a linked list. Turns out that array is not an efficient data structure in all cases. Arrays are implemented in programming languages in a certain way which makes it very efficient for certain cases while they are highly inefficient for other cases.

Given how arrays are stored in memory, it is usually inefficient to insert elements in the middle.

However, linked list also has drawbacks. You cannot access an element randomly and have to traverse from the first node to the next and so on until you find your desired node.

> *If you need to access elements at random positions in the list, don't use a Linked List.*

The Node element in JavaScript

As shown above , a Linked List consists of Node elements. The below code snippet shows the typical code structure for a Node element.

```
var node = {
  data: 0,       // some data value
  next: null // reference to next node
}
```

You can see that the Node element consists of two fields:

1. Data field
2. Reference to next Node

Initially the next field will point to null . As we create more nodes in the chain, we will update the next reference to point to the next node in the linked list.

First Take - Manually Build a Linked List

First Take - Manually Build a Linked List

Okay, let's look at a very simplistic example of how we can implement Linked Lists in JavaScript. We will create three nodes as shown in the above diagram. We'll then traverse the linked list and print the values on the console.

> Run the code below and see how it works

JavaScript

HTML

CSS (SCSS)

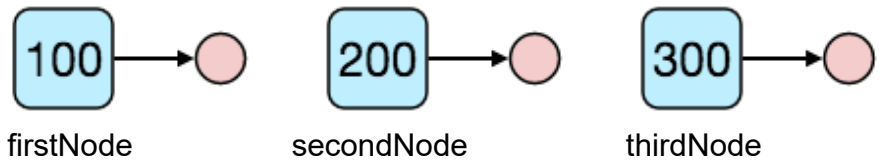
```
var firstNode = {
  data:100,
  next: null
};

var secondNode = {
  data: 200,
  next: null
};

var thirdNode = {
  data:300,
  next: null
};

firstNode.next = secondNode;
secondNode.next = thirdNode;

var currentNode = firstNode;

// Iterate over the Linked List and print nodes
while (currentNode != null) {
  console.log("Node value: " + currentNode.data);
  currentNode = currentNode.next;
}
```

Console

Clear

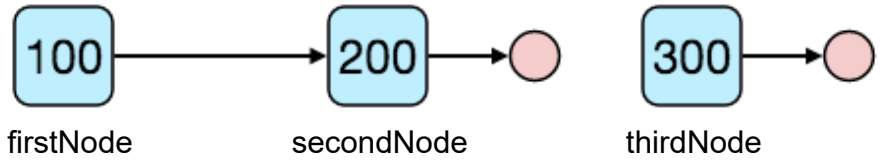
```
Node value: 100
Node value: 200
Node value: 300
```

The below animation shows the steps that we followed to create the linked list



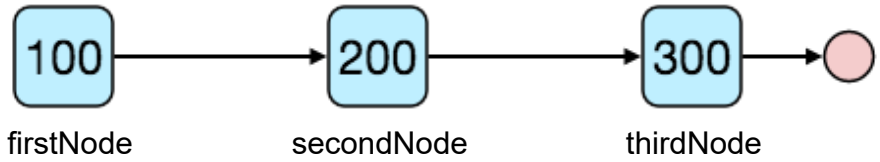
First, we create 3 nodes.
Next references for all nodes point to *null*.

1 of 3



Connect firstNode to secondNode.

2 of 3



Connect secondNode to thirdNode.
thirdNode is the last node and keeps pointing to null.

3 of 3

Let's take a quick quiz.

Quiz #1

Q

What is the general declaration of a node in a linked list

Check Answers

Generalizing Linked List Operations

It's obvious from the above code that if we wanted to create multiple nodes, we will have to write the same lines of code over and over again (to iterate over the linked list, find the last element and extend the linked list by adding a new node). Instead, we will create a LinkedList class that provides such functions over the linked list.

The below code snippet shows how linked lists can be generalized in JavaScript.

> Run the code below and see how it works

JavaScript

HTML

CSS (SCSS)

```
function Node(data) {
  this.data = data;
  this.next = null;
}

function LinkedList() {
  this._length = 0;
  this._head = null;
}

LinkedList.prototype.push = function(data) {
  // Create a new node with Data
  var node = new Node(data);
```

```
  // We are inserting the first node in the list
  if (this._head === null) {
    this._head = node;
  } else {
    // Find the last node
    var current = this._head;

    while (current.next) {
      current = current.next;
    }

    current.next = node;
  }

  // Increment the length
  this._length++;
}

// We follow the 0 based indexes just like Arrays
LinkedList.prototype.itemAt = function(index) {
  // Ensure that the index is within bounds
  if (index < 0 || index >= this._length) {
        // Return Null when index is out of bounds
        return null;
  }

  var current = this._head;
  var counter = 0;

  while (counter < index) {
    current = current.next;
    counter++;
  }

  return current.data;
}

// Returns Size of Current Linked List
LinkedList.prototype.size = function() {
  return this._length;
}

// Let's create a Linked List and add 3 nodes
var list = new LinkedList();
list.push(100);
list.push(200);
list.push(300);

for (i = 0; i < list.size(); i++) {
  console.log("Node value: " + list.itemAt(i));
}
```

Console

Clear

```
Node value: 100
```

```
Node value: 200
```

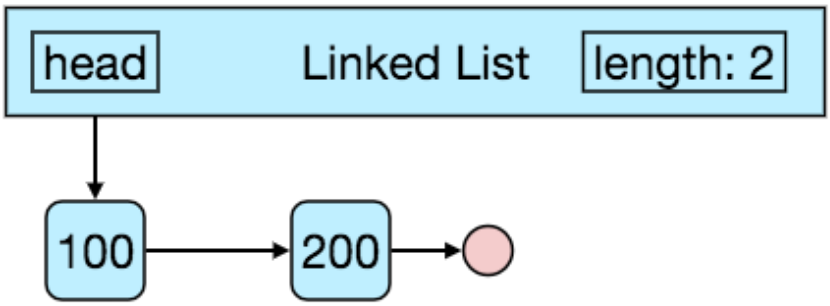
The below animation shows what the code does to create a linked list

Initial state of linked list object.
Head points to *null* and length is 0.

1 of 4

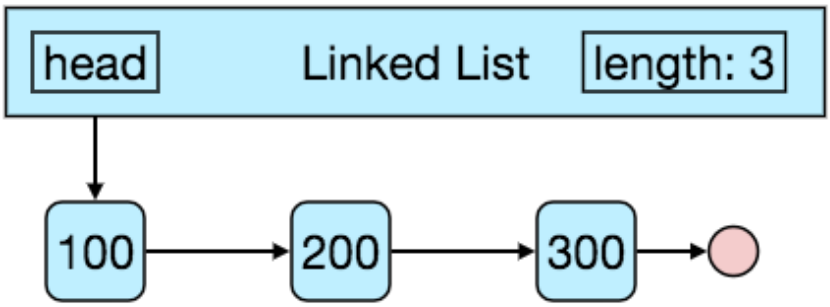
Inserting the first node with vaue 100.
Head points to the first node and length is 1.

2 of 4



Inserting the second node with vaue 200.
First node points to second node and length is 2.

3 of 4



Inserting the third node with vaue 300.
Second node points to third node and length is 3.

4 of 4

Taking a closer look at the Linked List Code

Feel free to skip to Quiz #2 below if you've already understood the above code.

The first section of code created our *Node* class. The function takes the data value , assigns it to the node via the 'this' attribute. Initially, the function will cause the node to point to nothing because at this point in time we don't know where the node needs to be inserted in the linked list.

```
function Node(data) {
  this.data = data;
```

```
  this.next = null;
}
```

Next we defined our linked list. The linked list will initially have a length of *0* and the first element or the head element will point to nothing. Simple enough.

```
function LinkedList() {
  this._length = 0;
  this._head = null;
}
```

Next we add a *push* function to our linked list class. This function will be used to append nodes to the linked list. There are two cases while appending a node to the linked list.

1. If the linked list is empty, then we are inserting the first node. This node will become the head node.
2. Linked List already has one or more nodes. In this case, node will be inserted at the tail of the linked list.

We create the node and then insert the node at the right position. Note how we traverse to the last element in the linked list. We have a temporary variable called *current* which is updated in the loop to the next element in the list. When *current.next* becomes null, it means that we have reached the end of the list. Here, the while loop will terminate. We point the *next* of the last node to the newly created node.

As the last step, we increment the size of the linked list.

```
LinkedList.prototype.push = function(data) {
  // Create a new node with Data
  var node = new Node(data);

  // We are inserting the first node in the list
  if (this._head === null) {
    this._head = node;
  } else {
    // Find the last node
    var current = this._head;

    while (current.next) {
      current = current.next;
    }
```

```
    current.next = node;
  }

  // Increment the length
  this._length++;
}
```

Similar to the *push* method, we introduced *itemAt* method. This method retrieves and returns the value of node at the given index. It's similar to array's lookup method. As the first step, we ensure that the Linked List has enough elements. If index is too low (less than 0) or too high (greater than the length), we simply return null.

We could have considered throwing an exception here as well (many programming languages throw OutOfBoundException is such cases).

Now, if the linked list has enough items, we traverse to the desired index, starting from the head node. Remember what we discussed in the beginning as a LinkedList v.s. Array tradeoff. In Array, we could have randomly accessed a given index. Here, we have to iterate through the linked list in a serial order until we get to the desired node. Once we are at the node, we return the data contained by that node.

```
// We follow the 0 based indexes just like Arrays
LinkedList.prototype.itemAt = function(index) {
  // Ensure that the index is within bounds
  if (index < 0 || index >= this._length) {
        // Return Null when index is out of bounds
        return null;
  }

  var current = this._head;
  var counter = 0;

  while (counter < index) {
    current = current.next;
    counter++;
  }

  return current.data;
}
```

Quiz #2 - Generalized Linked List

1 What does the code line this.head normally refer to

2

When adding a node to a linked list there is normally a special case condition which is identified by the code snippet if (this._head === null){ this._head = node;}. What does this code snippet actually do?

Check Answers

Removing a Node from the Linked List

Let's see how to remove nodes from a linked list. Our remove function will take the index of the element to be removed. If the node to be removed happens to be the head node, it will update the head node as well.

> Run the code below and see how it works.

JavaScript

HTML

CSS (SCSS)

```
// Removes the element and returns the data
// in the node that was removed
LinkedList.prototype.remove = function(index) {
  // Ensure that the index is within bounds
```

```
  if (index < 0 || index >= this._length) {
        return null;
  }

  var current = this._head;

  if (index === 0) {
        // Special case for removing the head node.
    this._head = current.next;
  } else {
        // Track previous element
    var previous = null;
    var counter = 0;

    while (counter < index) {
      previous = current;
      current = current.next;
      counter++;
    }

    // Set previous node's next
    // to the node after the deleted node
    previous.next = current.next;
  }

  this._length--;
  return current.data;
}

// Let's create a Linked List and add 3 nodes
var list = new LinkedList();
list.push('Stacks');
list.push('Queues');
list.push('Arrays');
list.push('Sets');

console.log("Length before removal: " + list.size());

// Remove the 3rd element
var removed = list.remove(2);
console.log("removed: " + removed);

// Remove the head node
var removed = list.remove(0);
console.log("removed: " + removed);

console.log("Length after removal: " + list.size());
```

Console

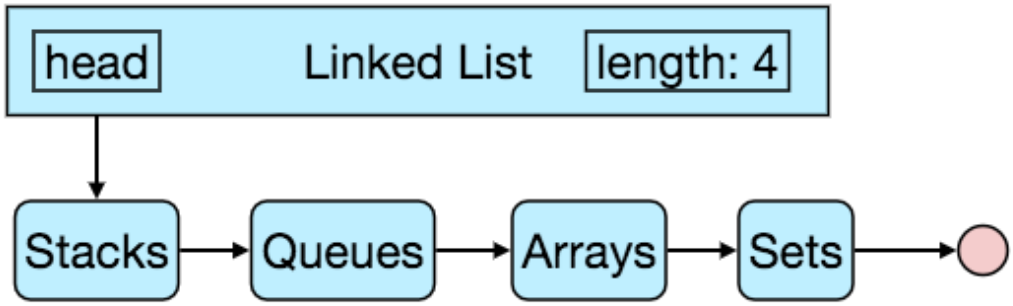
Clear

```
Length before removal: 4
removed: Arrays
```

```
removed: Stacks
```

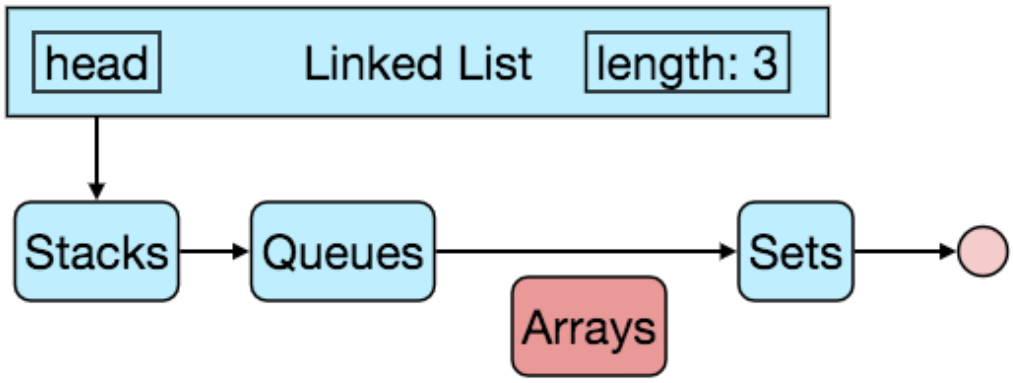
```
Length after removal: 2
```

Below visualization shows how nodes were removed in the code.



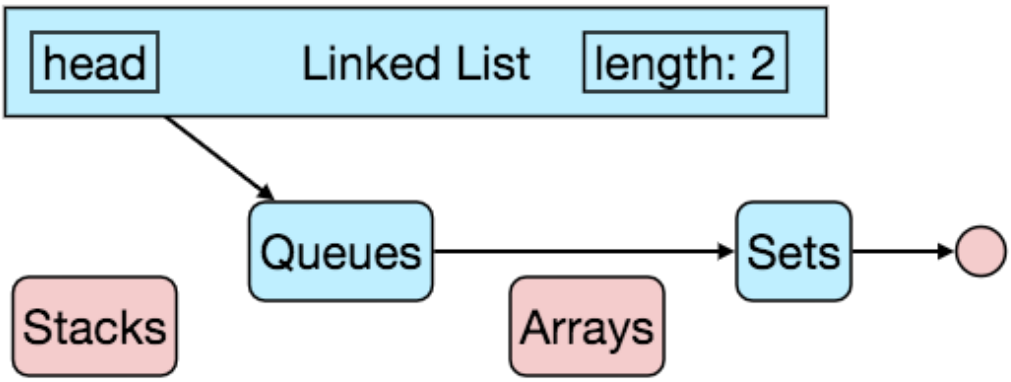
Linked List before any element is removed

1 of 3



Removing node at Index 2.
Node at Index 1 now points to Node at Index 3

2 of 3



Removing node at Index 0.
With current head node removed, head now points to the new head.

Let's go through the deletion code. Feel free to skip to Quiz #3.

We start at the head and traverse to the node to be deleted. If the node to be deleted is the head, then we just point the *_head* to the second node in the list (which could be null if it's a single node linked list).

If the node to be deleted is not the head, then we traverse to the node that is going to be deleted. While traversing to the node, we keep track of the previous node as well. Why? Because in a singly linked list, there is no way to access the previous node of a given node. Hence, we need to explicitly track the previous node as we traverse. Once we reach the desired node, we connect the *previous node's next* to *current node's next*. Think of it as if the current node was a middleman and now previous node is making a direct connection to the node after the current node (or metaphorically speaking, we are removing the middleman).

After setting the references, we then reduce the length and return the data in the current node.

Here's the code for deletion.

```
// Removes the element and returns the data
// in the node that was removed
LinkedList.prototype.remove = function(index) {
  // Ensure that the index is within bounds
  if (index < 0 || index >= this._length) {
        return null;
  }

  var current = this._head;

  if (index === 0) {
        // Special case for removing the head node.
    this._head = current.next;
  } else {
        // Track previous element
    var previous = null;
    var counter = 0;

    while (counter < index) {
      previous = current;
      current = current.next;
      counter++;
    }

    // Set previous node's next
    // to the node after the deleted node
    previous.next = current.next;
  }

  this._length--;
  return current.data;
}
```

Quiz #3 - Removing nodes from a linked list

1 Why do we track the previous node while removing

2 What should the previous.next point to if we just removed the last node?

Check Answers

Exercise

Let's do an exercise. We now want to implement the search in a linked list. Search function takes in a value and either returns the index of the node if the value exists or returns null if value doesn't exist in the linked list.

> Run the following code. Implement search method so that the tests pass.

JavaScript

HTML

CSS (SCSS)

```
// Returns the index of the node containig
// the data (assume unique data for simplicity)
LinkedList.prototype.search = function(data) {
  /////// TODO ///////
  ////// IMPLEMENT THIS /////
  ////// RUN AFTER IMPLEMENTING THIS /////
  ////// TESTS SHOULD PASS /////

  return undefined;
}

// This is the function that runs test cases.
runEvaluation();
```

Console

Clear

```
*** There is some bug lurking there. See failed test cases ***
```

```
Here are the tests that ran:
```

```
Here are the tests that fail:
```

```
Test case FAILED for list.search(100). Result: undefined. Expected: null
```

```
Test case FAILED for list.search(81). Result: undefined. Expected: 9
Test case FAILED for list.search(49). Result: undefined. Expected: 7
Test case FAILED for list.search(16). Result: undefined. Expected: 4
```

Summary

- Linked list comprises of a chain of nodes.
- Each node contains a data element and the reference to the next node in the chain.
- Linked lists are better for scenarios where you need to insert or remove elements from the middle.
- Unlike arrays, linked lists don't provide random access. Hence, look up and retrieval in linked lists are slower than an array.

Binary Trees & Binary Search Trees

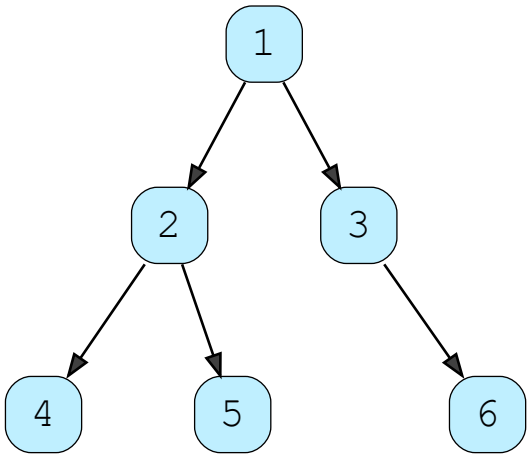
Introduction

A binary tree is a linked data structure where each node points to two child nodes (at most). The child nodes are called the ***left child*** and ***right child***.

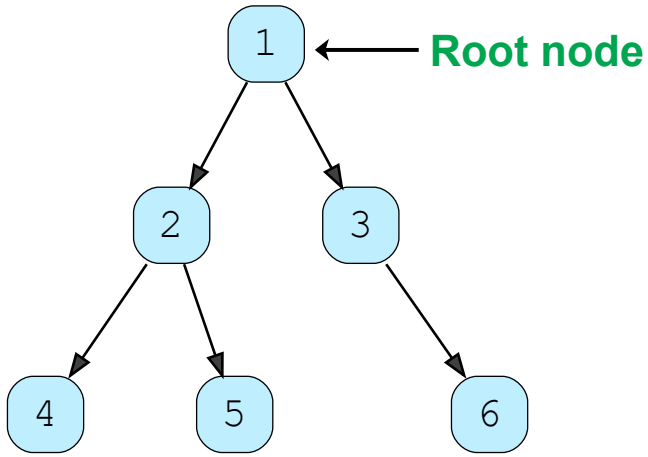
Unlike other linear data structures that we looked, it's a hierarchical data structure. Here are the properties of a binary tree.

1. Each node can point to two children at most.
2. The top most element in the tree is called ***root***.
3. Two Children are usually referred as ***Left Child*** and ***Right Child***.
4. The nodes which don't point to any children are called ***leaf nodes***.
5. Non-leaf nodes are called ***internal nodes***. Root is also an internal node if it's not a leaf node.

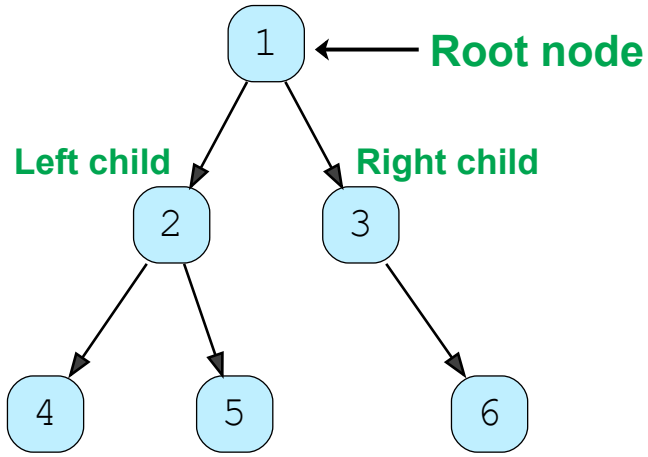
Let's step through a few visualizations to internalize these properties.



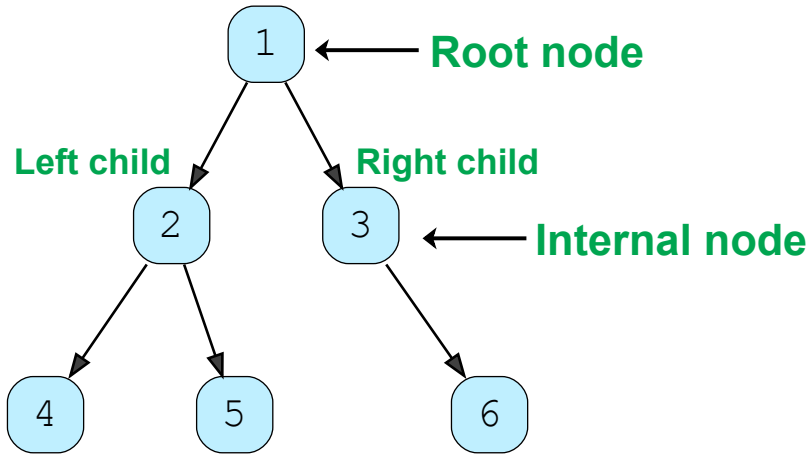
Here's our binary Tree



Node with value 1 is the root node

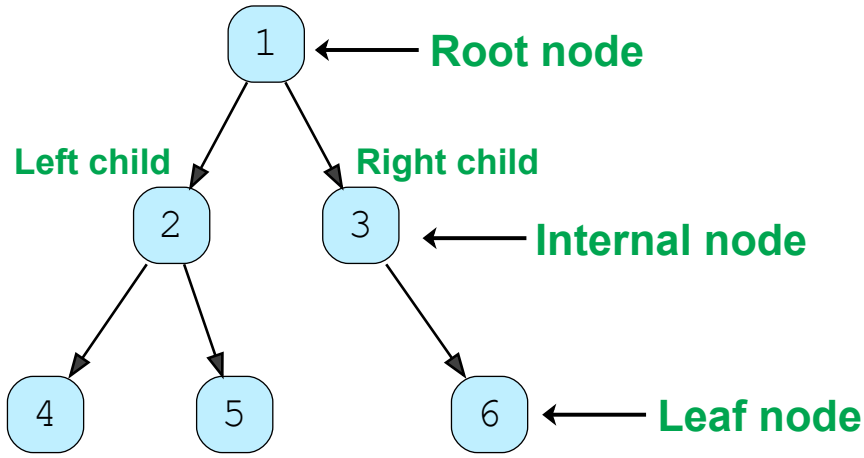


Nodes with value 2 & 3 are root's children



Nodes with value 2, 3 are internal nodes

Nodes with value 2, 3 are internal nodes



Nodes with value 4, 5, 6 are leaf nodes

Tree Basics Quiz

1 If a Tree has only 1 node, what type of node is it (select all that apply)

2 If a Tree node has one child, what type of node is it?

Check Answers

Height and Depth of the Tree

Now let's look at a couple more properties of the nodes and tree that are very useful for the discussion on trees.

The ***Depth of the node*** is the number of nodes on the path from the root to the node. Root has depth 0.

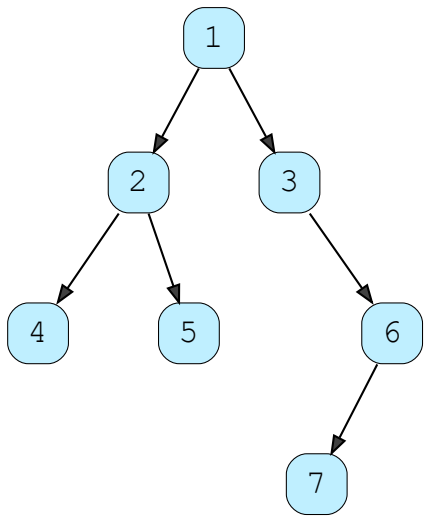
The ***Height* of the node** is the number of nodes on the path from the node to the root. All leaf nodes have height 0.

You can see that for depth, we begin counting from the root, down towards to the node, starting at 0. For height, we start counting from the node, up towards the root, again starting at 0. Some people start counting depth and height from 1 instead of 0 and it's just a matter of preference.

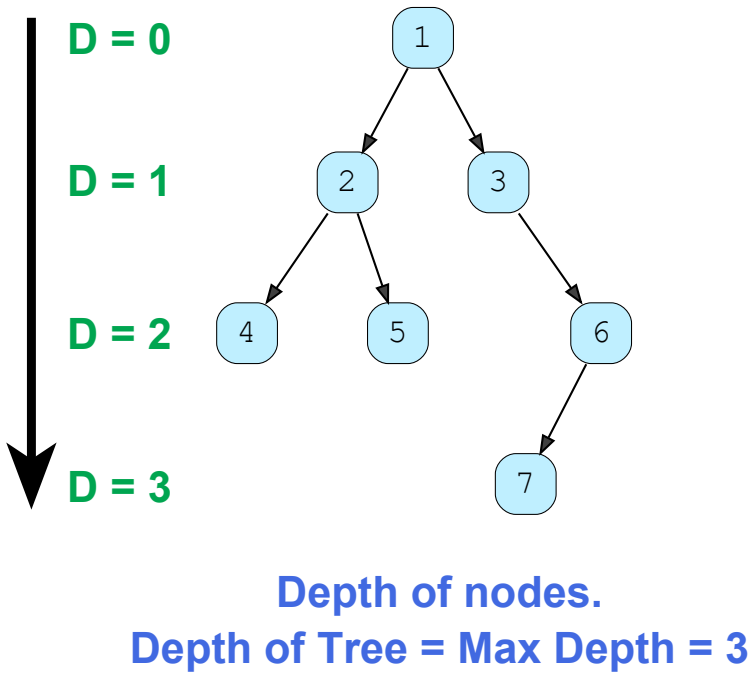
The ***Height of the tree*** is the maximum height of any node in the tree.

The ***Depth of the tree*** is the maximum depth of any node in the tree.

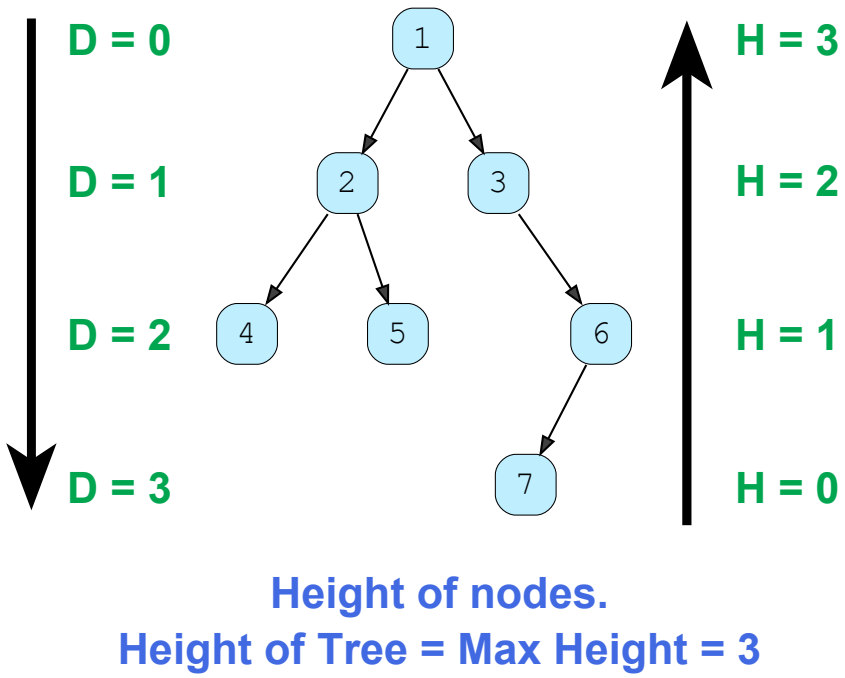
Hence, depth of the tree is always equal to the height of the tree. Let's step through a visualization to look at the height and depth properties.



1 of 3



2 of 3



3 of 3

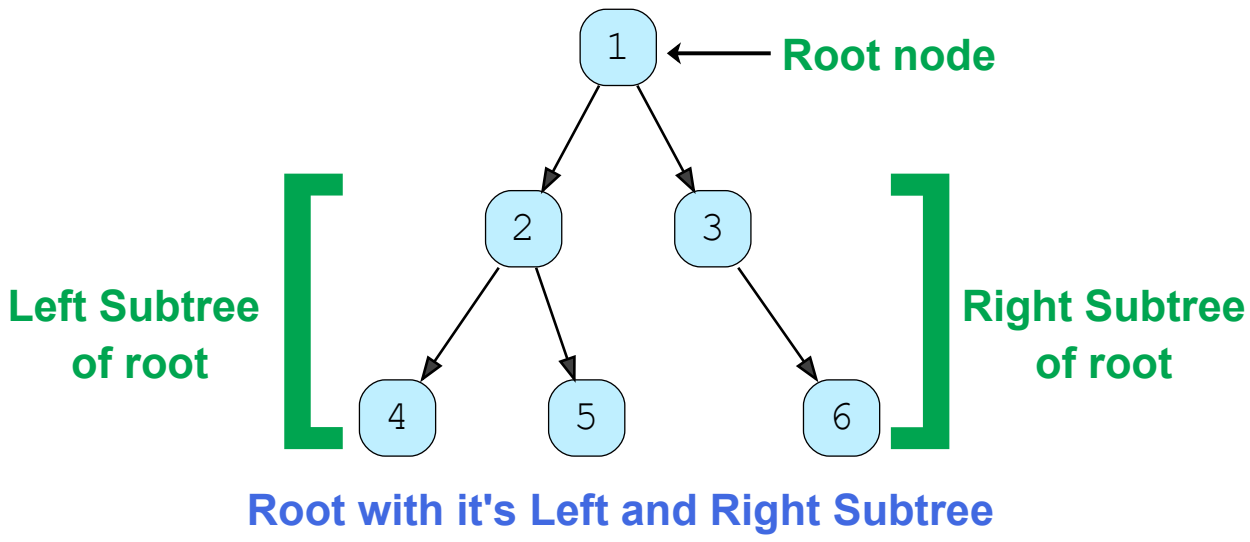
Key of the node and Subtree

In a binary tree, the node might contain a lot of data with a key. Our examples are a little simplistic as we are only working with a single integer which acts

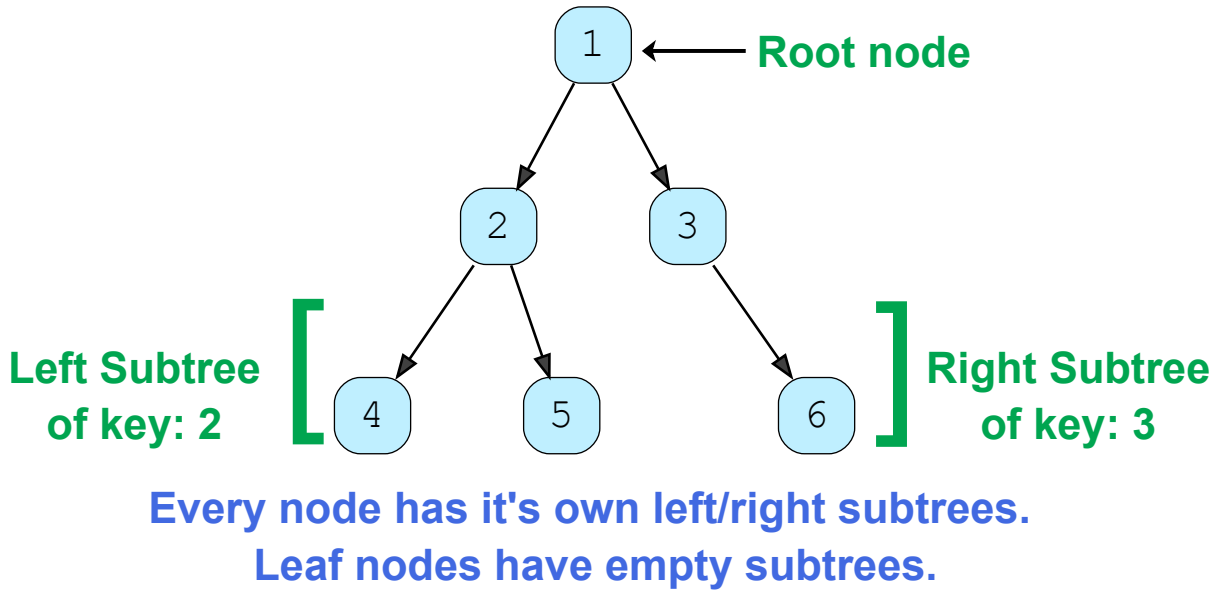
both as a key and value. However, in many practical situations, a node contains several fields of data with a key. In our examples below, we will continue to use a single value as data and key, but it's a good thing to remember that, in practice, a node might contain more data.

A subtree of the node is the tree formed by its left and right children. Each node in a binary tree has a left subtree and a right subtree.

Let's step through the following visualization to understand the subtrees.



1 of 2



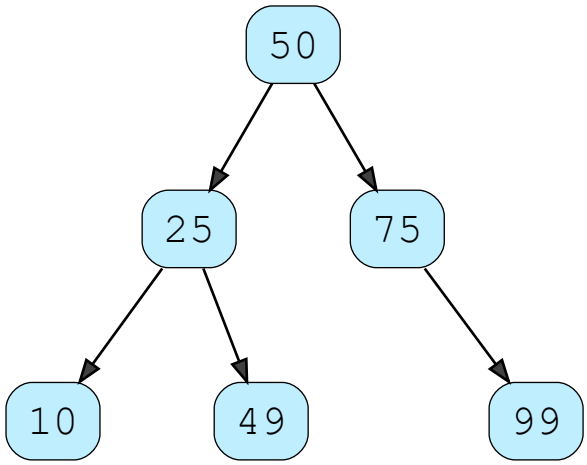
2 of 2

Binary Search Tree

The most common binary tree is called a Binary Search Tree. Why are we discussing a specialized form of Binary Tree without even getting to the code? Because the structure and layout of data in a Binary Tree would make more sense to you if we discuss insertion/deletion/lookup in a Binary Search Tree.

So what's a Binary Search Tree (BST hereafter)? The name already gives a decent hint. It's a binary tree that helps in searching the data in the tree. We'll shortly look at how a BST facilitates efficient lookups. To understand this, first let's understand the properties of a BST.

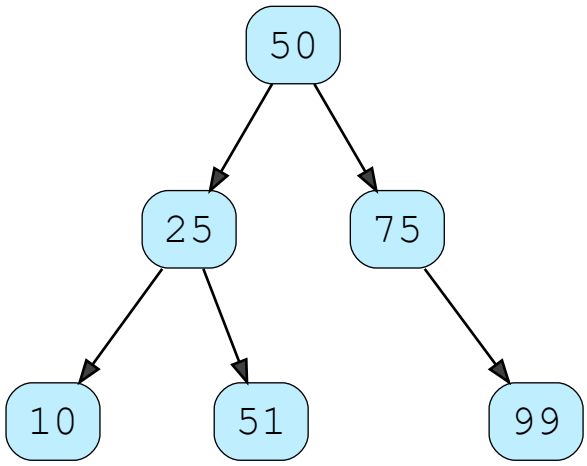
A binary tree is a BST if the key of the node is greater than ***all*** the nodes in its left subtree and is smaller than ***all*** the nodes in its right subtree. Let's look at a binary search tree.



A Binary Search Tree

Let's take a closer look at the above BST. Root has the key *50*. Look at all the nodes in its left subtree. They are all less than *50*. Now, look at the right subtree of the root. All the keys in the right subtree are greater than *50*. Hence, it's a BST.

Let's look at another example and see whether it's a BST or not.



Is it a BST?

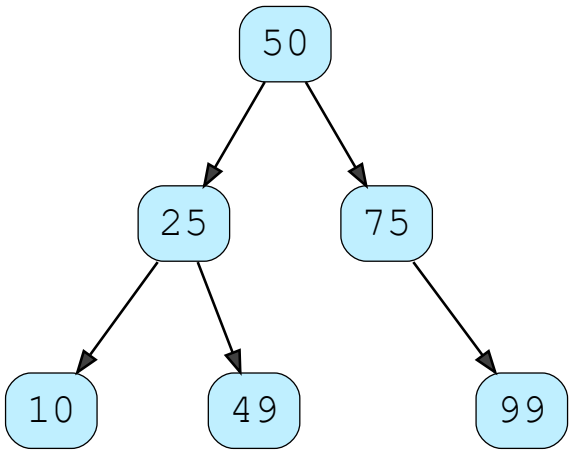
The tree above is *not* a BST. Why? Because of the leaf node containing *51*. It's in the left subtree of *50* which breaks our invariant.

Search in a Binary Search Tree

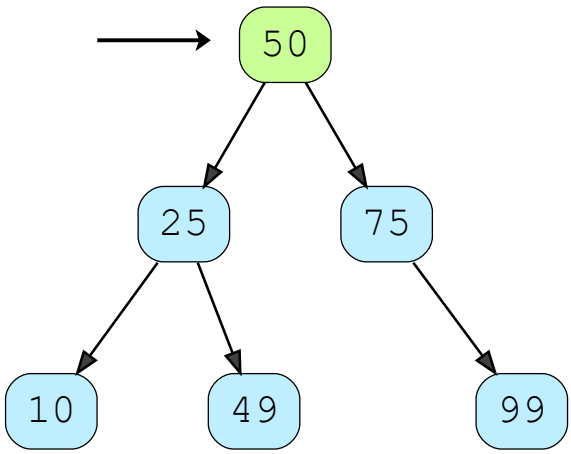
Now that we know what a BST looks like, let's see how can it help with lookups. It's pretty simple actually. Suppose you are looking for key X. Here's what you do at each step starting at the root.

1. Compare current node's key with X. If it's equal, we've found the key. All done.
2. If X is less than node's key, we start looking at node's left subtree. It's because we know that right subtree cannot contain anything greater than X.
3. If X is greater than node's key, we start looking in the right subtree.
4. We repeat this process until we find the key or we reach the leaf node. If we reach the leaf node and haven't found the key as yet, we return not found.

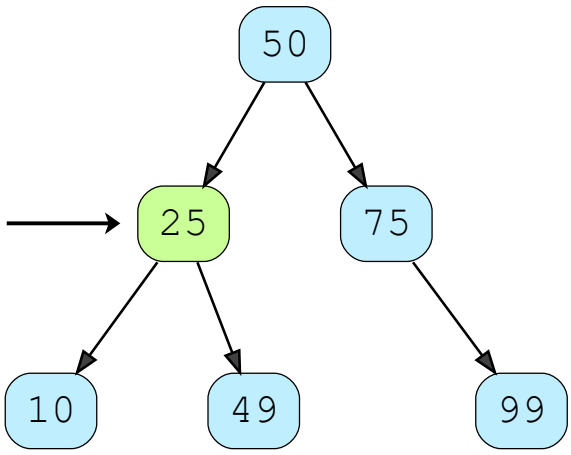
Let's look at search in a BST in action.



Let's search for key 49



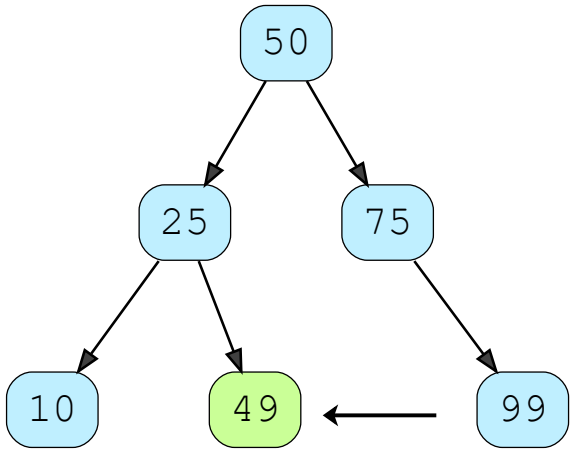
We start the search at root.
Root is 50 (> 49) so we go the left subtree.



25 is less than 49.

We go to the right subtree.

3 of 4



We found node containing 49

4 of 4

Notice that tree has six elements but we only needed to do three comparisons before we found our desired node. If you look closely, you can see that at each step, we eliminate almost half of the tree.

One thing to remember here is that this is the best case for a BST. There are many cases where your BST might not give you an optimal performance, but they are beyond the scope of this introduction.

We have seen a lot of theory regarding Binary trees and Binary Search trees. Let's look at some code.

Defining the Tree Node

We'll define a Node class with a data field and two fields to point to the left and the right child.

```
function Node(data) {
  this.data = data;
  this.left = null;
  this.right = null;
}
```

Defining Binary Search Tree

The following code defines how the tree would be represented in Javascript.

```
function BST() {
  this._root = null;
}
```

Inserting into BST

In a BST, we need to find the correct location for the Node to be inserted. It depends on the value of the key. Assume, we want to insert a node with key K. Starting at the root, here's a quick description of how we are going to locate the correct location.

1. Compare current node's key with K.
2. If K is less than the current node,
    1. If left child of current node is Null, we insert K as the left child of current node and return.
    2. If the left child is not Null, the left child becomes the new current node, and we repeat the process from step 1.
3. If K is greater than the current node,
    1. If right child of current node is Null, we insert K as the right child of the current node and return.
    2. If the right child is not Null, the right child becomes the new current node, and we repeat the process from step 1.

Using the steps described above, we insert K at the correct location so that BST is quickly searchable later on. Let's step through a quick visualization to see insertion in action.

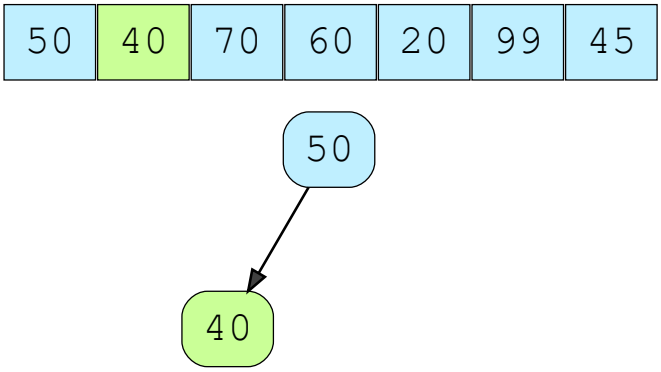
50	40	70	60	20	99	45

We'll insert the list into BST

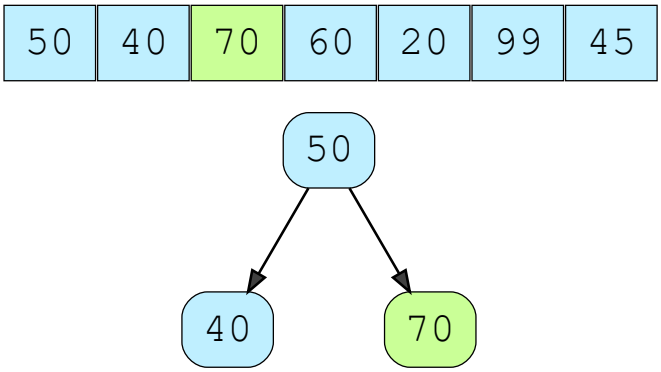
50	40	70	60	20	99	45

50

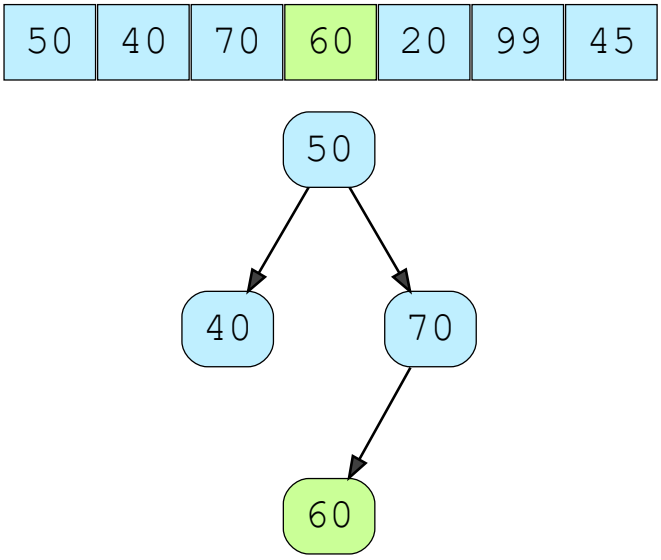
Insert 50 as root



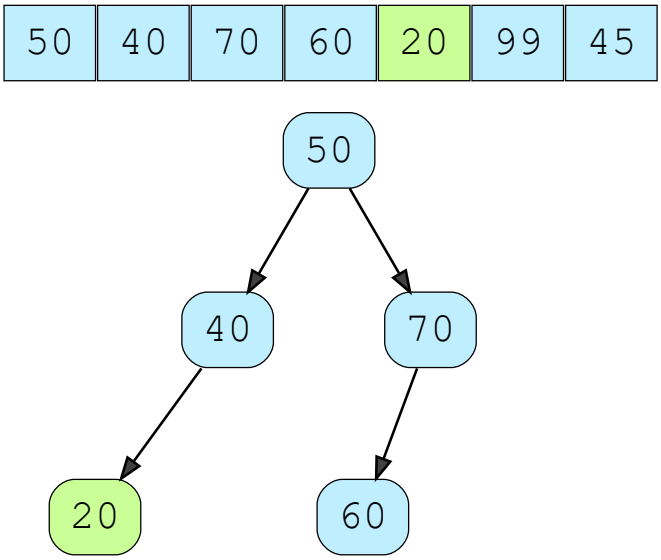
Insert 40



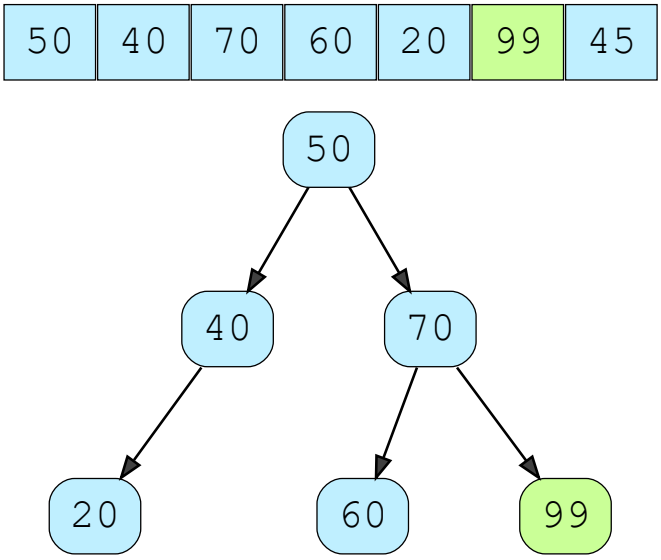
Insert 70



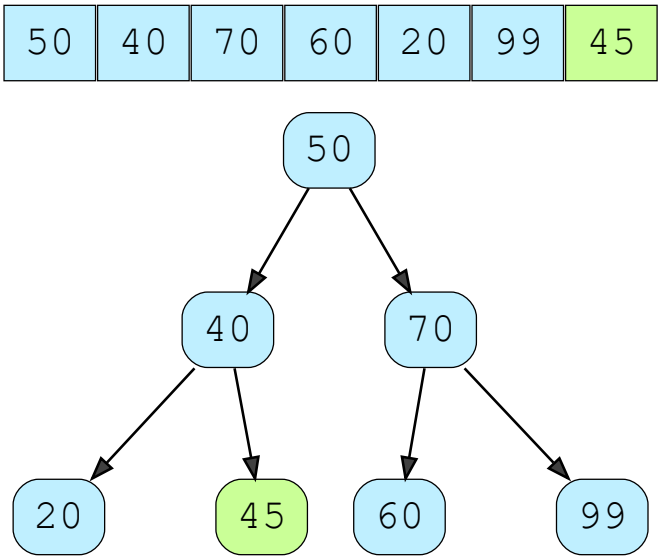
Insert 60



Insert 20



Insert 99



Insert 45.
... and we're done.

You might be wondering what do we do in case of a duplicate. For now, we simply ignore it. There are some binary trees in which we can allow

duplicates but if you are searching/sorting on a Key, then duplicated keys don't make sense in many other scenarios.

Let's look at the insertion code for a BST.

```
BST.prototype.insert = function(data) {
  var node = new Node(data);

  // If it's the first node
  if (this._root === null) {
    this._root = node;
    return;
  }

  var current = this._root;

  while (current) {
    if (data < current.data) {
      if (current.left === null) {
        current.left = node;
        return;
      }
      current = current.left;
    } else if (data > current.data) {
      if (current.right === null) {
        current.right = node;
        return;
      }
      current = current.right;
    } else {
      // Duplicates are not supported
      return;
    }
  }
};
```

Implementing Search in a BST

We've already described how lookup works in Binary Search Tree. Let's look at the code for the lookup method as well. It returns True if the key is found and returns false otherwise.

```
BST.prototype.contains = function(data) {
  var current = this._root;

  while (current) {
    if (data === current.data) {
          return true;
    }

    if (data < current.data) {
      current = current.left;
    } else {
      current = current.right;
```

```
    }
  }


  return false;
};
```

BST Insert and Search in action

> Run the following code to see BST insert and search.

JavaScript

HTML

CSS (SCSS)

```
var bst = new BST();
console.log('Insert into BST: 5');
bst.insert(5);

console.log('BST contains 5? Returns ' + bst.contains(5));
console.log('BST contains 6? Returns ' + bst.contains(6));
```

Console

Clear

```
Insert into BST: 5
BST contains 5? Returns true
BST contains 6? Returns false
```

Tree traversals

We've covered search in a BST. When you want to traverse all the elements in a tree, there are a few well-known techniques to do that. These methods define the ***order in which nodes are visited***, and each one of these traversal orders helps us solve different problems. Let's look at the three most common traversal techniques.

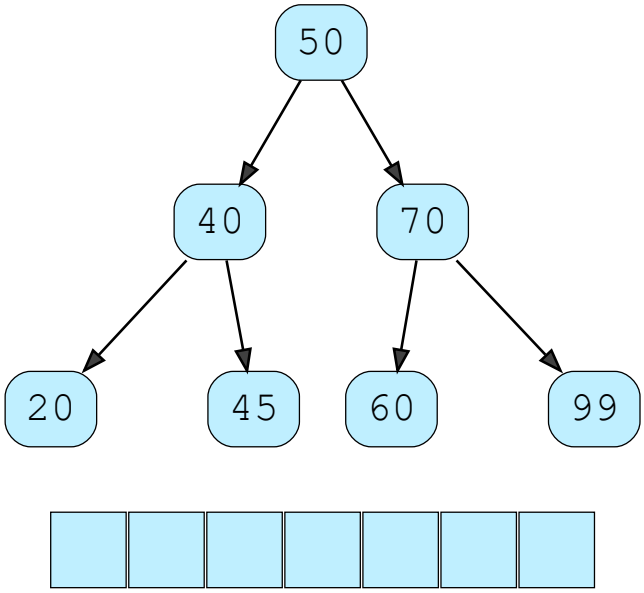
Pre-Order Traversal

In Pre-Order traversal, we traverse in the following order:

1. First visit the node itself.
2. Then visit the left subtree of the node.
3. Then visit the right subtree of the node.

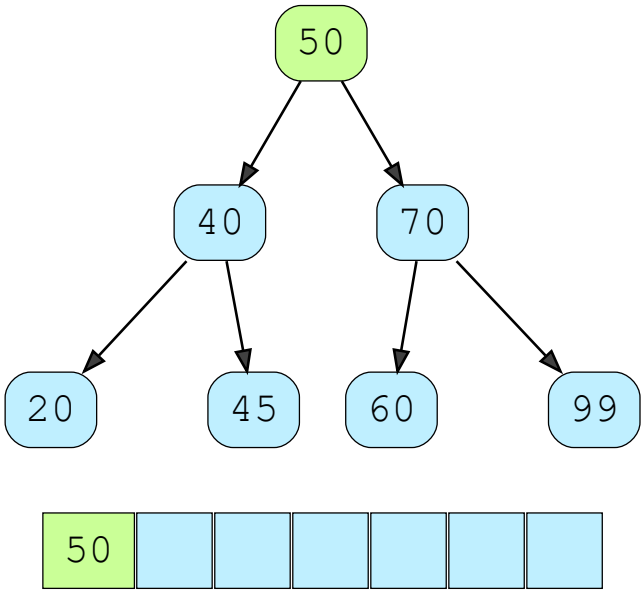
Let's step through a visualization to understand it better (Highly recommended to go to the last slide to see the complete order).

Let's traverse in Preorder

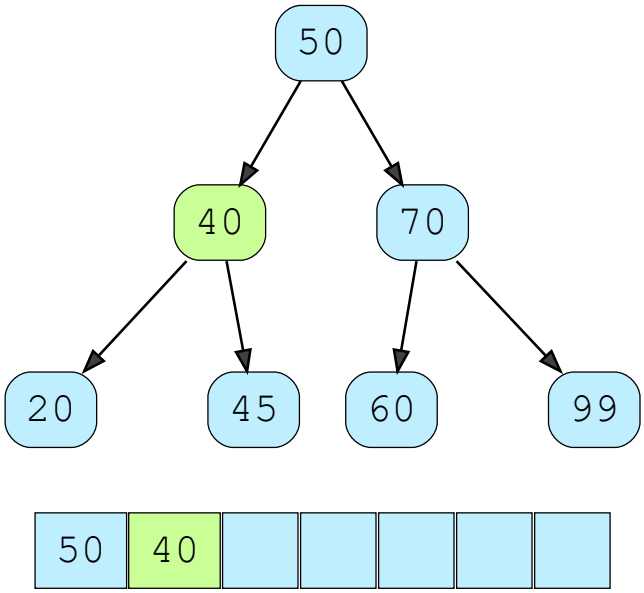


1 of 9

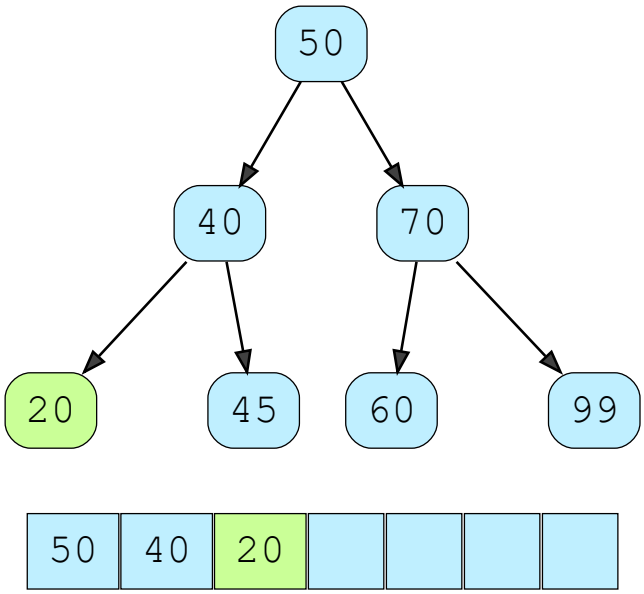
Start at Root 50. We visit it.



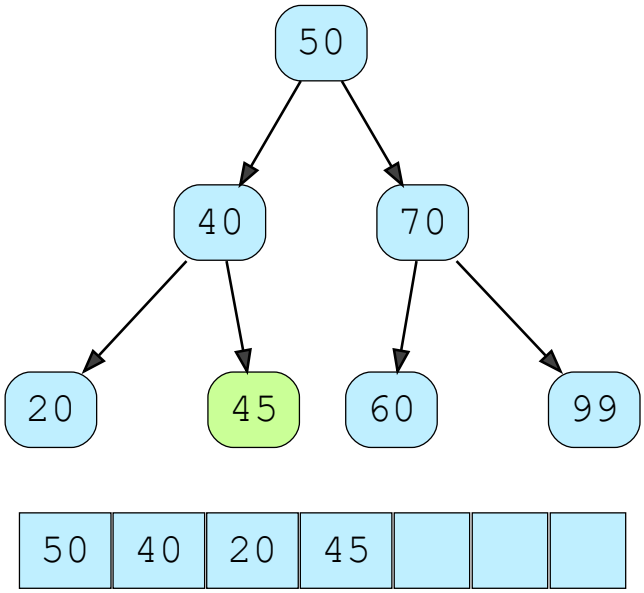
Traverse to left of 50. Visit it.



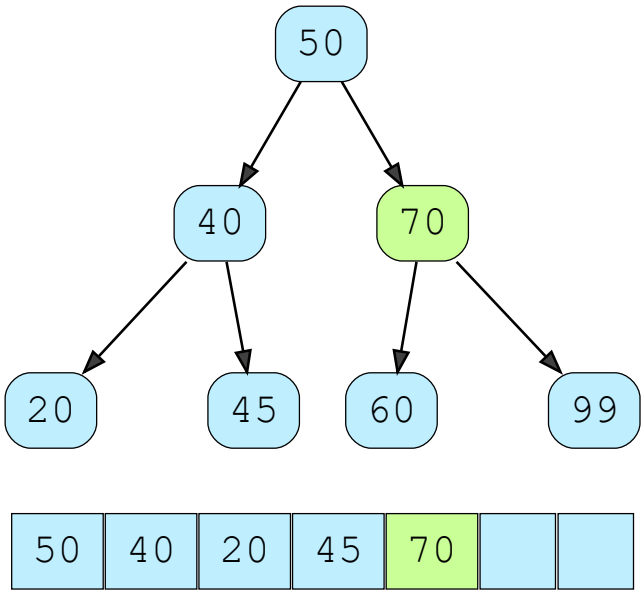
Traverse to left of 40. Visit it.



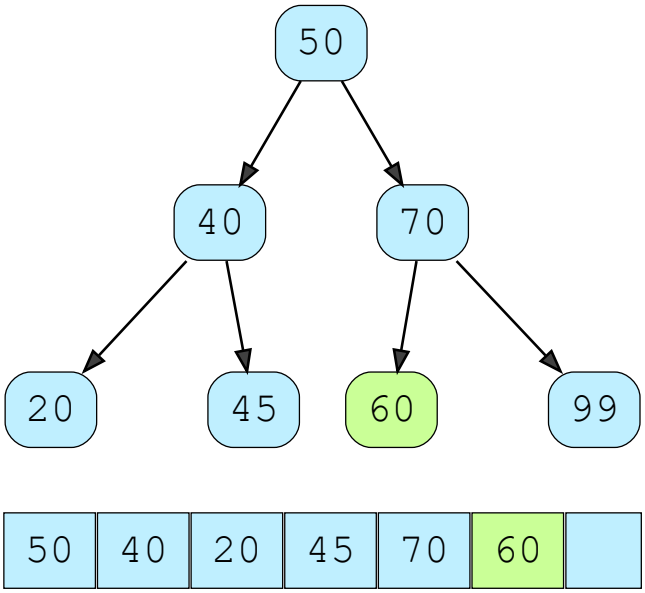
20 is leaf node. Visit right of 40



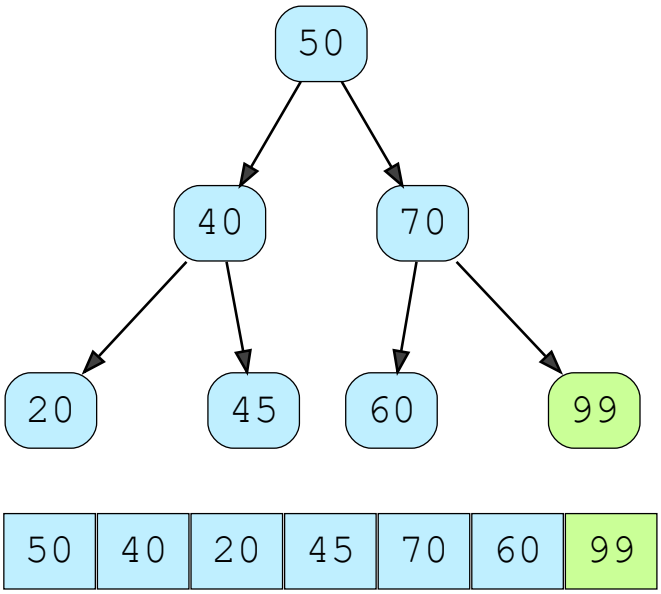
Done with left side.
Visit 50's right child 70.

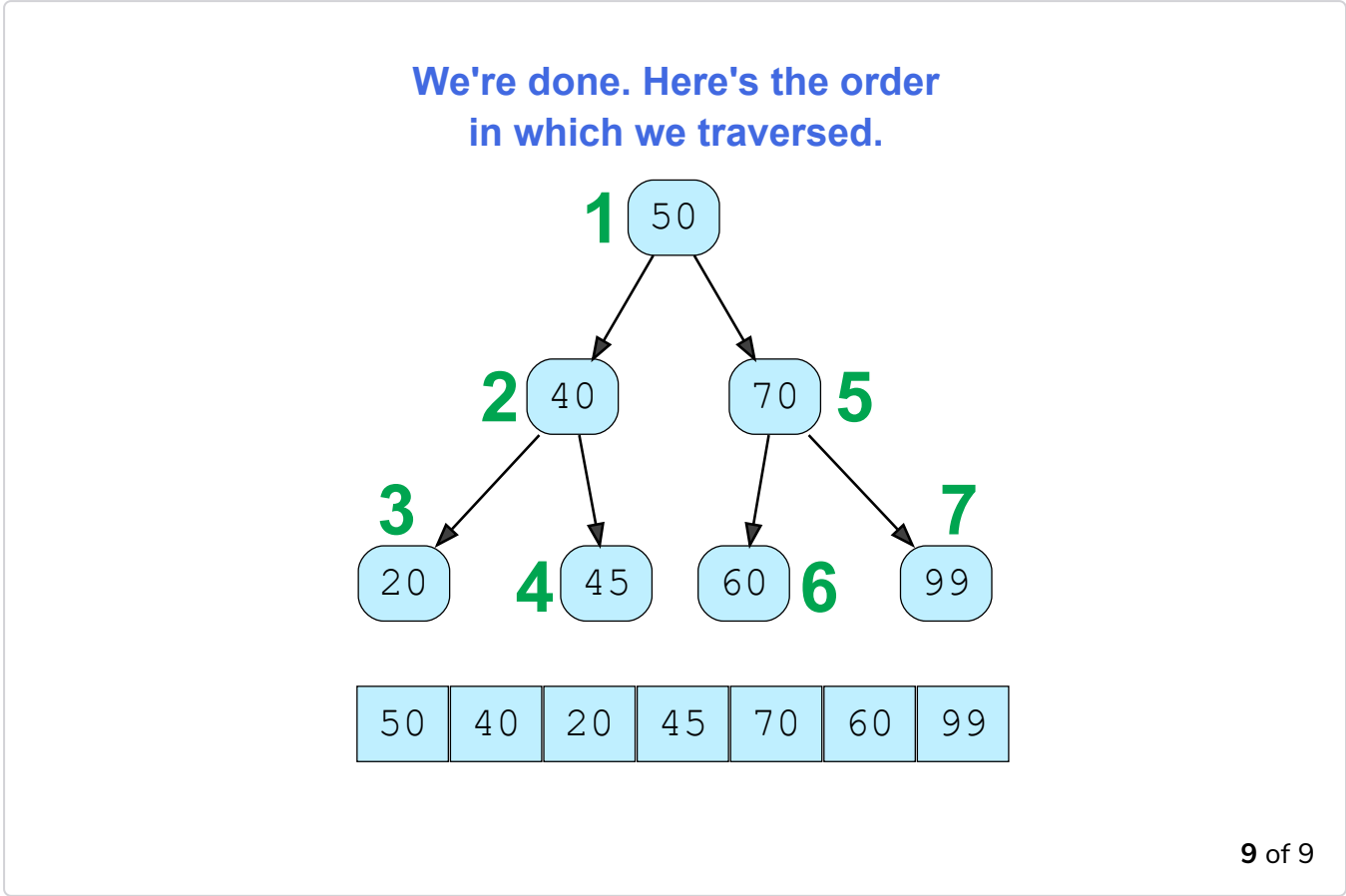


Visit 60 as it's left child of 70



Visit 70's right Child 99.





Tree traversals are implemented using recursion. Here's the code that fills the array in pre-order traversal.

```
// Returns keys in pre-order traversal
BST.prototype.preOrder = function() {
  var output = [];

  function preOrderImpl(node) {
    if (node === null) {
      return;
    }

    // Visit the node itself.
    output.push(node.data);

    // Visit left subtree
    preOrderImpl(node.left);

    // Visit the right subtree
    preOrderImpl(node.right);
  }

  // Call the internal function
  // with Root as the starting point.
  preOrderImpl(this._root);

  return output;
```

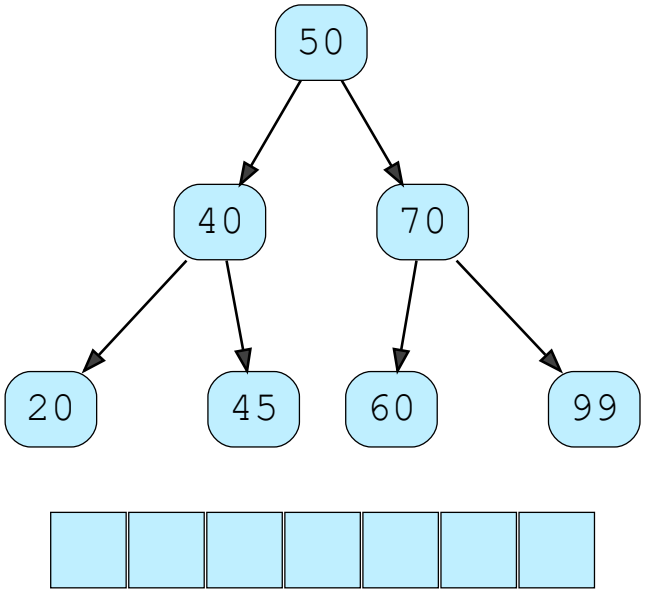
```
}
```

In-Order Traversal

In in-Order traversal, we traverse in the following order:

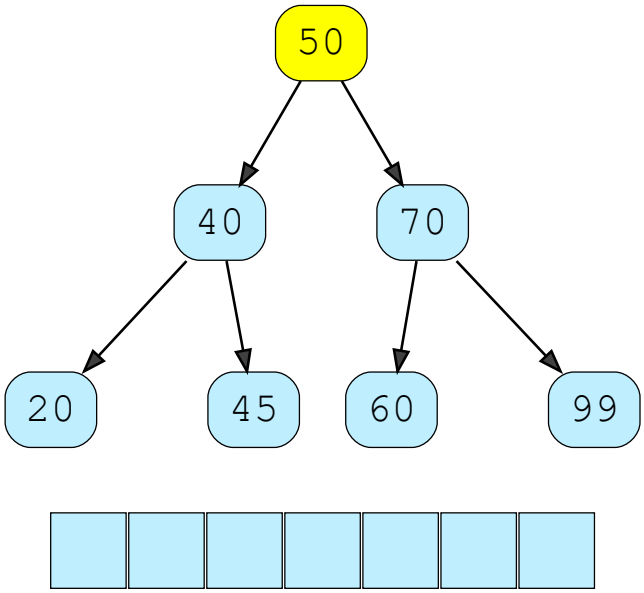
1. First visit the left subtree of the node.
2. Then visit the node itself.
3. Then visit the right subtree of the node.

Let's step through a visualization to understand it better (Highly recommended to go to the last slide to see the complete order).

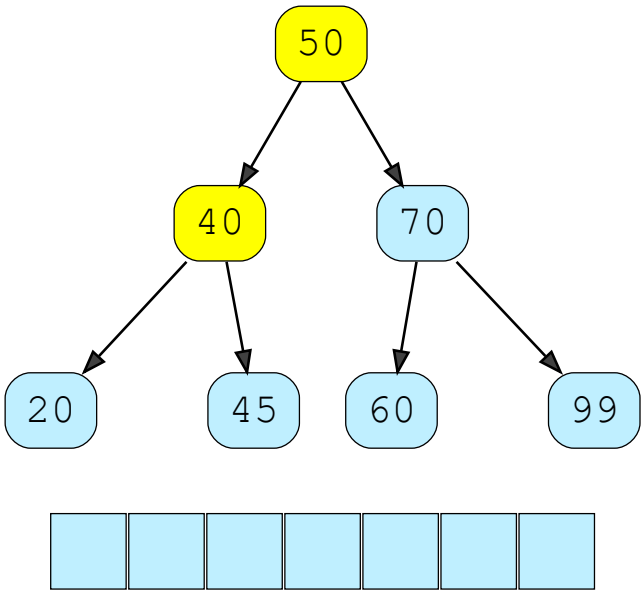


1 of 12

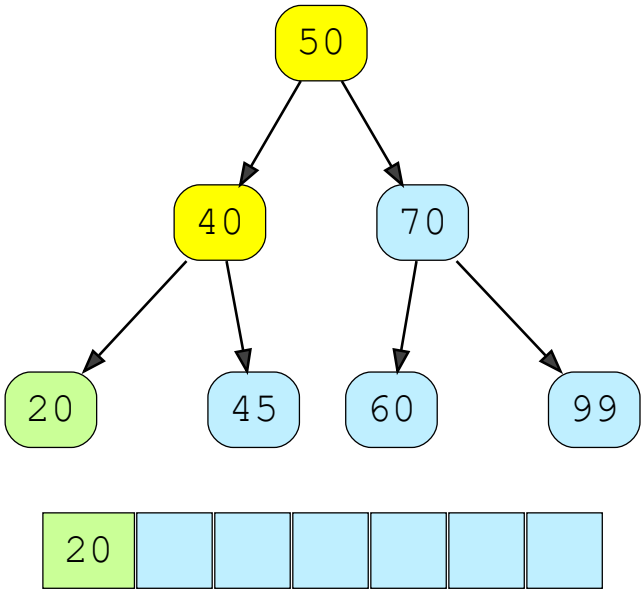
Start at Root 50. It has left subtree
Move to left child.



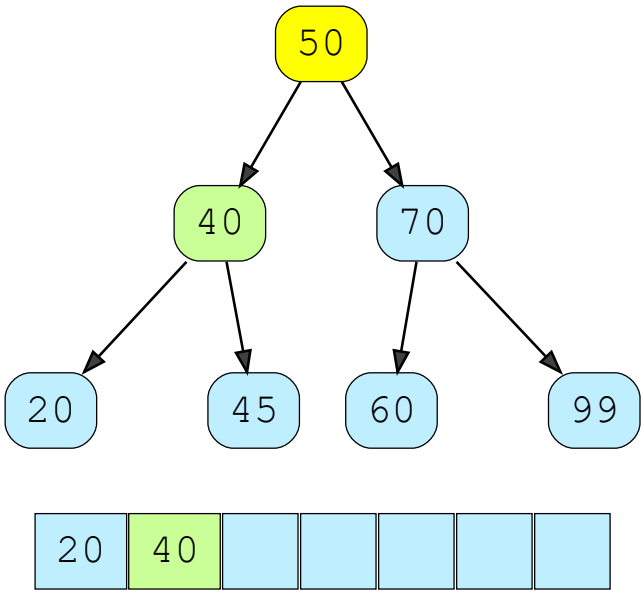
40 still has a left subtree.
Move to left child.



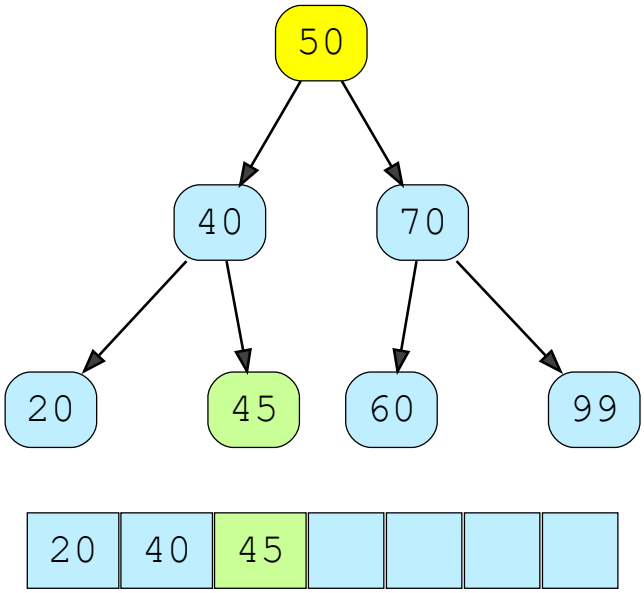
20 is a leaf node. Visit it.



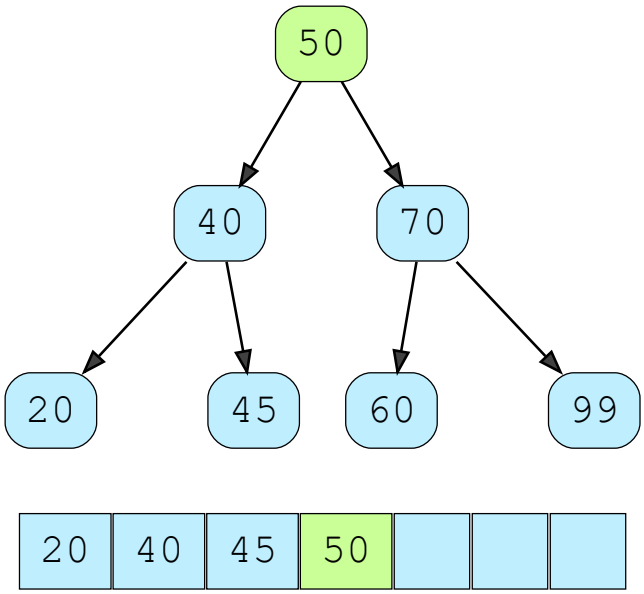
Done with 40's left subtree
Visit 40 itself.



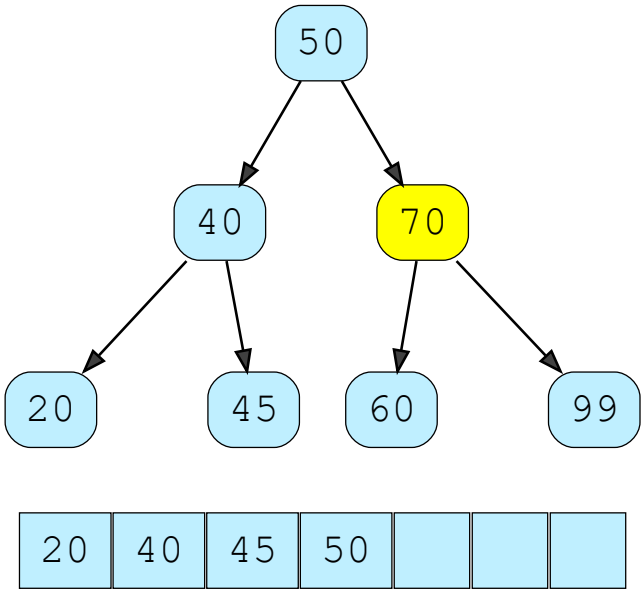
Start visiting 40's right subtree.
Visit 45.



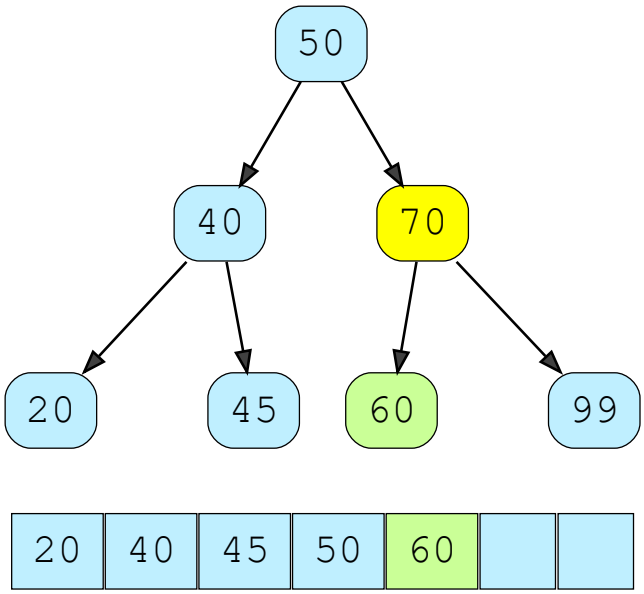
Done with visiting 50's left subtree.
Visit 50 itself.



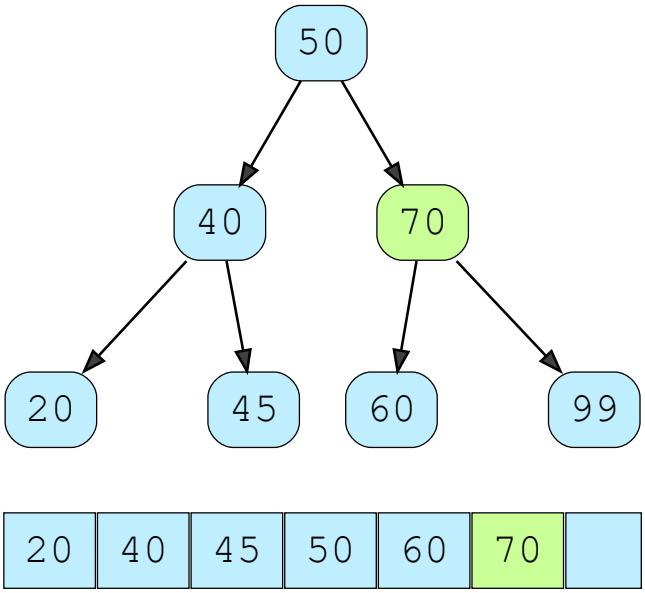
Start traversing 50's right subtree.
70 has left subtree so we go there.



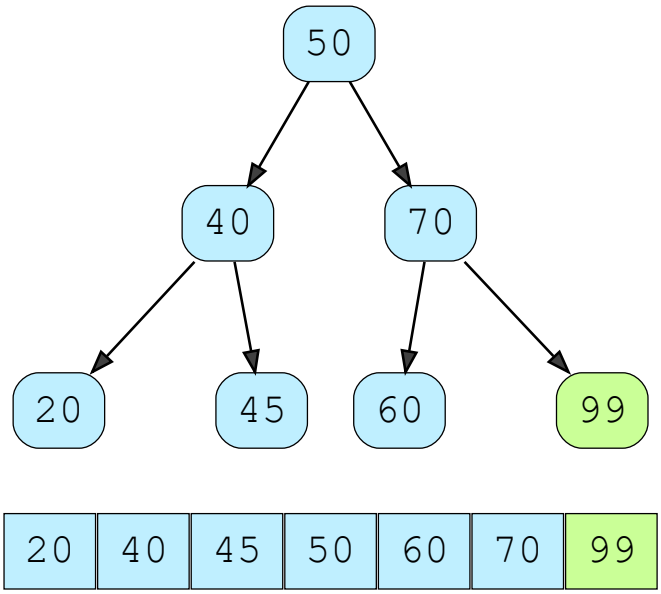
60 has no subtree.
Vist 60.

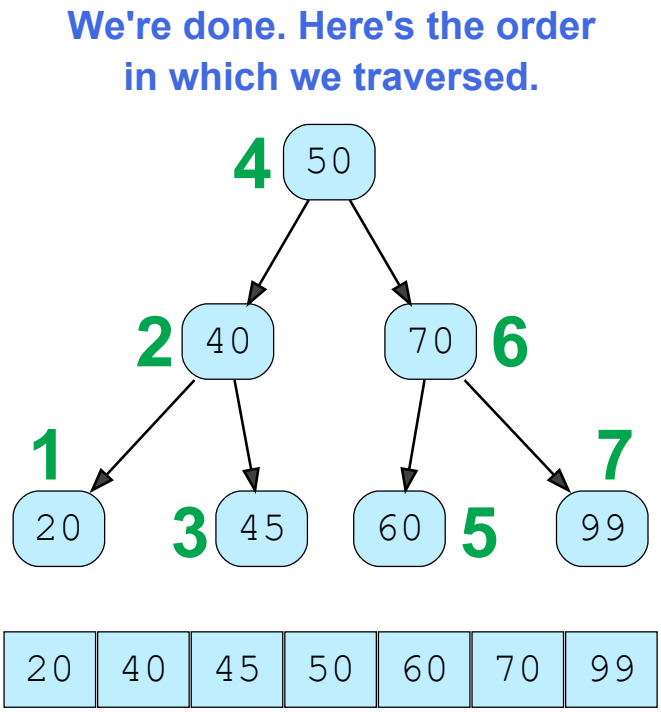


Done with 70's left subtree.
Visit 70 itself.



Visit 70's right subtree.
99 is leaf node. Visit it.





If you haven't noticed as yet - there is a nice *property* of In-Order traversal. It traverses the BST in ascending order. Now you know what traversal to use if you want to print/get all the nodes of tree in **sorted order.**

Let's look at the code for In-Order traversal.

```
// Returns Keys in the InOrder traversal
BST.prototype.inOrder = function() {
  var output = [];

  function inOrderImpl(node) {
    if (node === null) {
      return;
    }

    // Visit left subtree
    inOrderImpl(node.left);

    // Visit the node itself.
    output.push(node.data);

    // Visit the right subtree
    inOrderImpl(node.right);
  }

  // Call the internal function
```

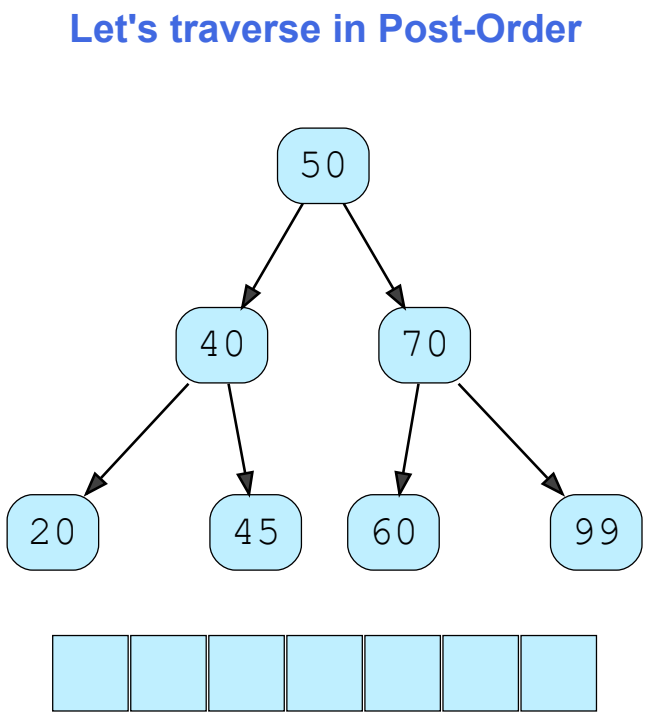
```
  // Call the internal function
  // with Root as the starting point.
  inOrderImpl(this._root);


  return output;
}
```

Post-Order Traversal

In Post-Order traversal, we traverse in the following order:

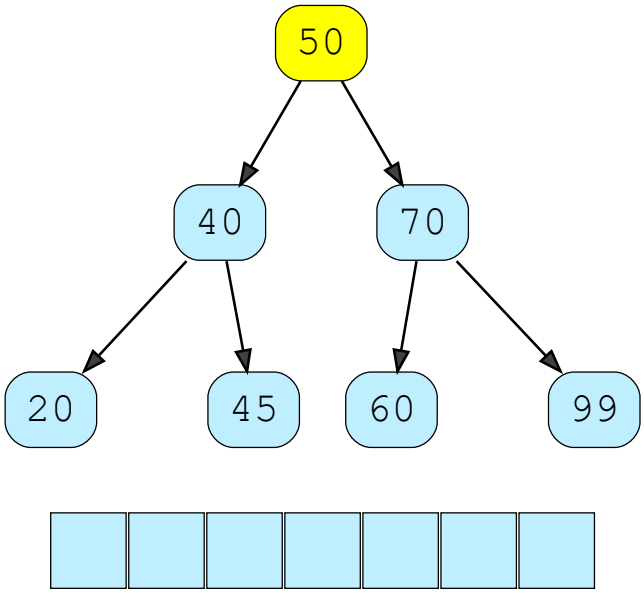
1. First visit the left subtree of the node.
2. Then visit the right subtree of the node.
3. Then visit the node itself.

Let's step through a visualization to understand it better (Highly recommended to go to the last slide to see the complete order).

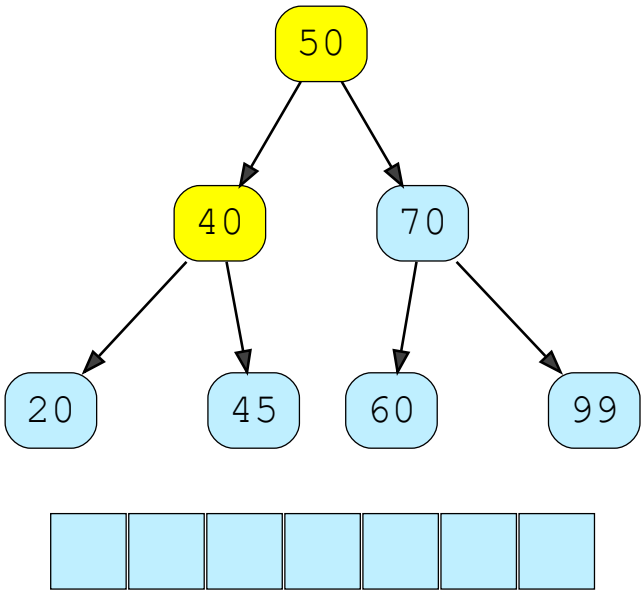


1 of 15

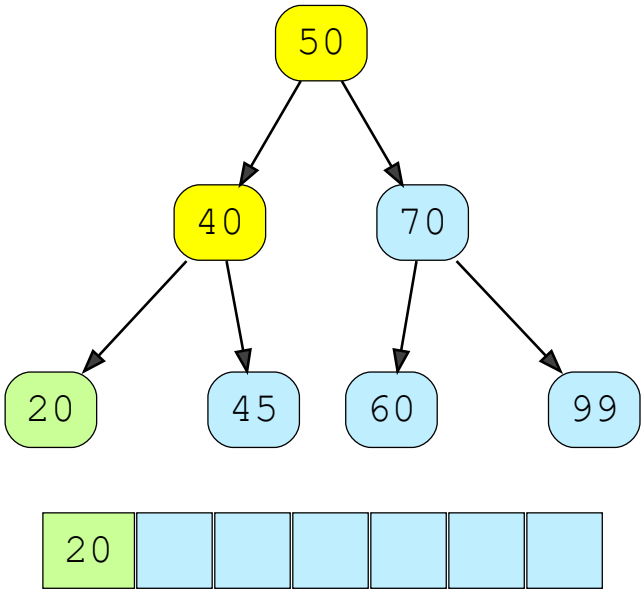
Start at Root 50. It has left subtree
Move to left child.



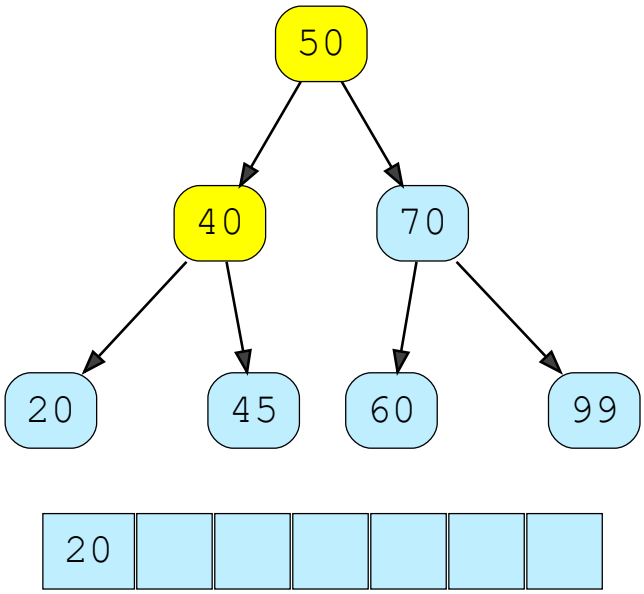
40 still has a left subtree.
Move to left child.



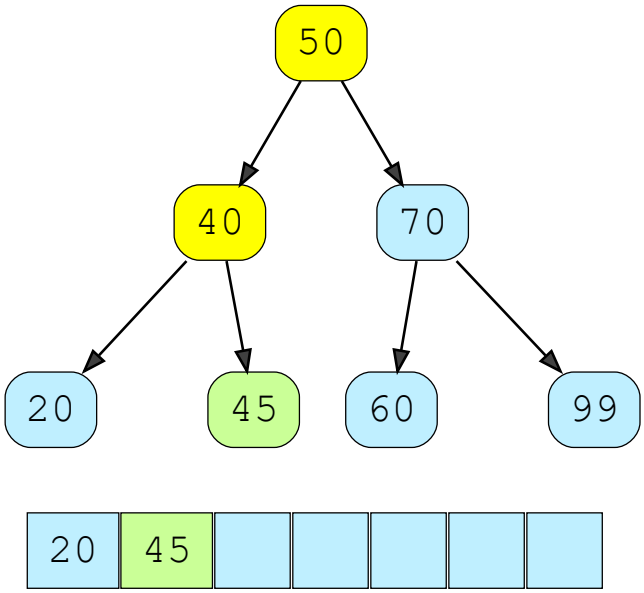
20 is a leaf node. Visit it.



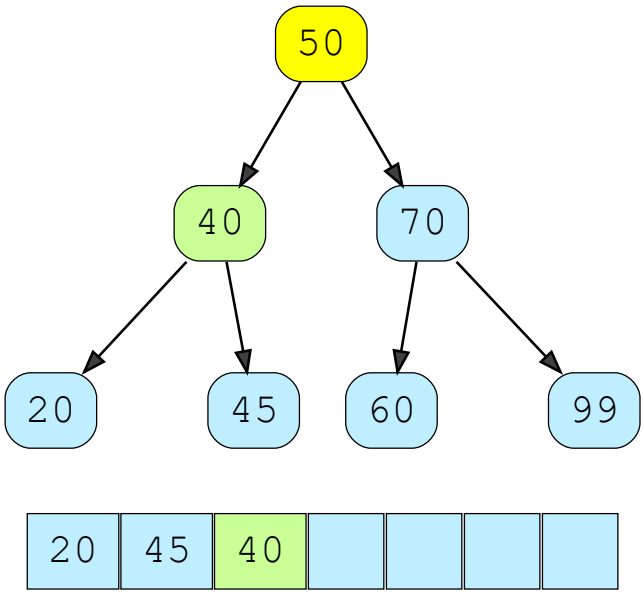
Done with 40's left subtree
Visit 40's right subtree.



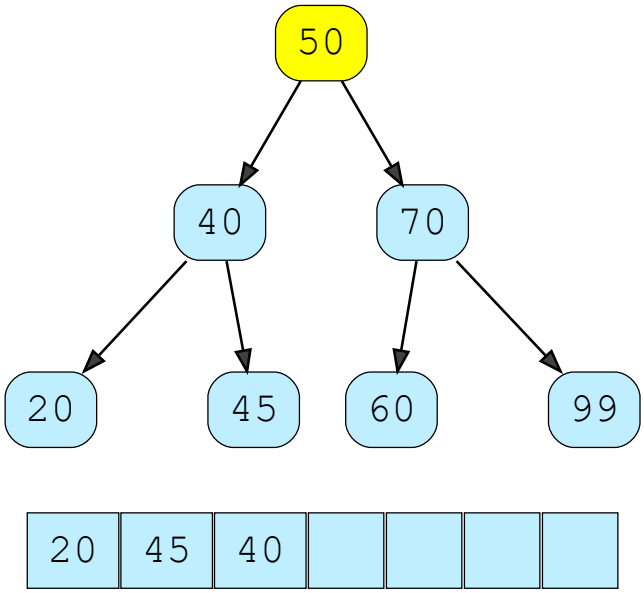
Start visiting 40's right subtree.
Visit 45.



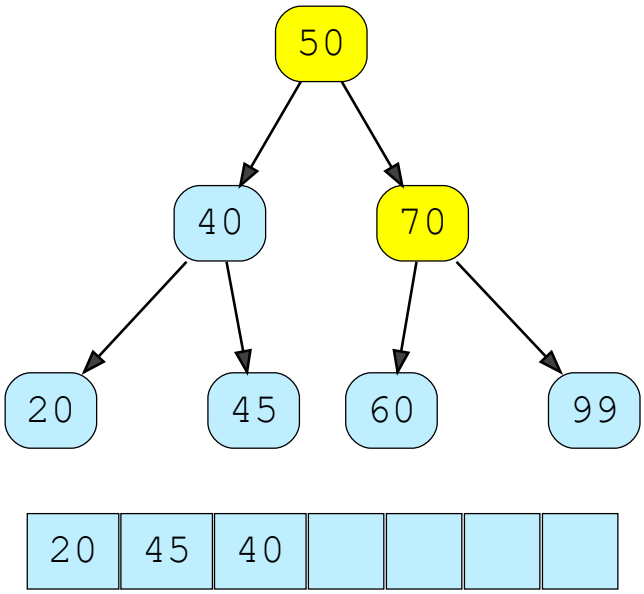
Done with visiting 40's both subtrees.
Visit 40 itself.



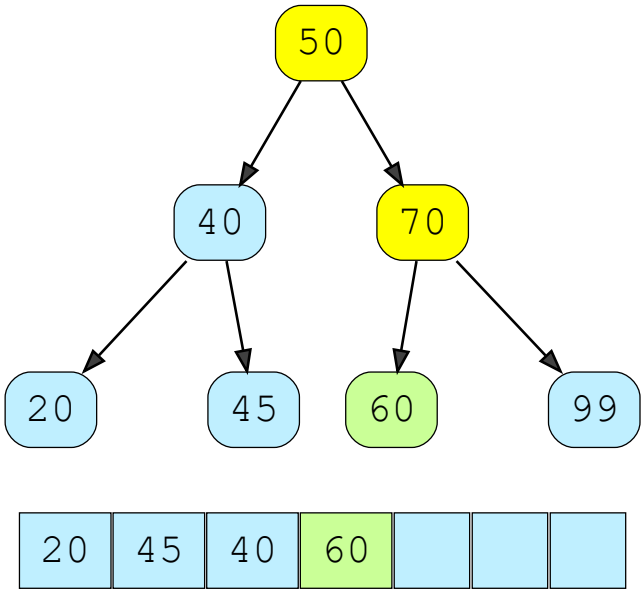
Done with 50's left subtree.
Start traversing 50's right subtree.



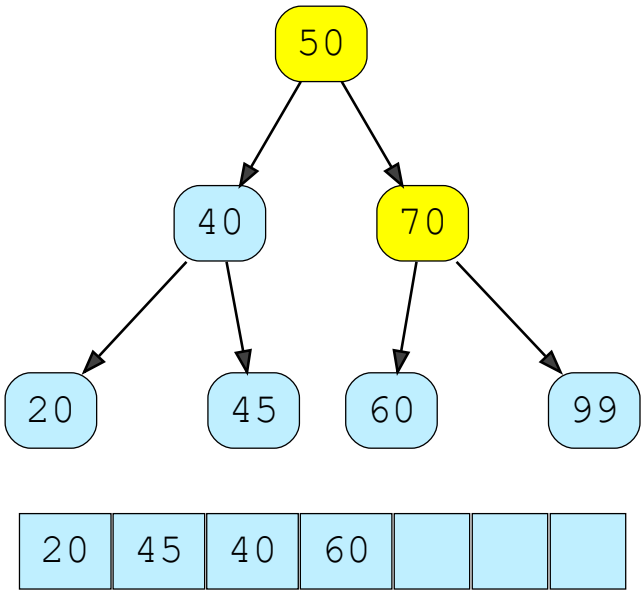
70 has left subtree.
Start traversing 70's left subtree.



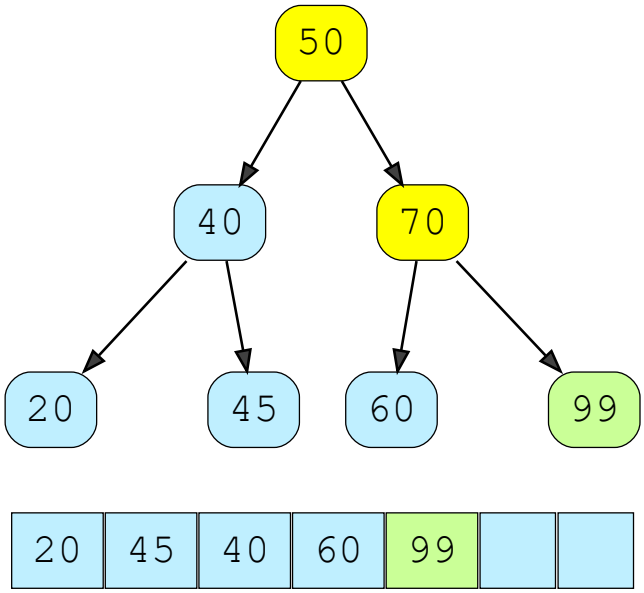
60 is leaf node. Visit it.



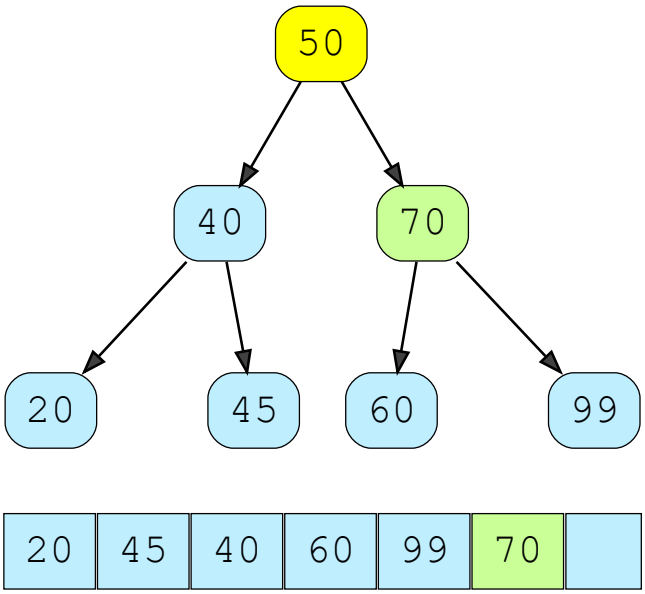
Done with 70's left subtree.
Start Visiting its right subtree.

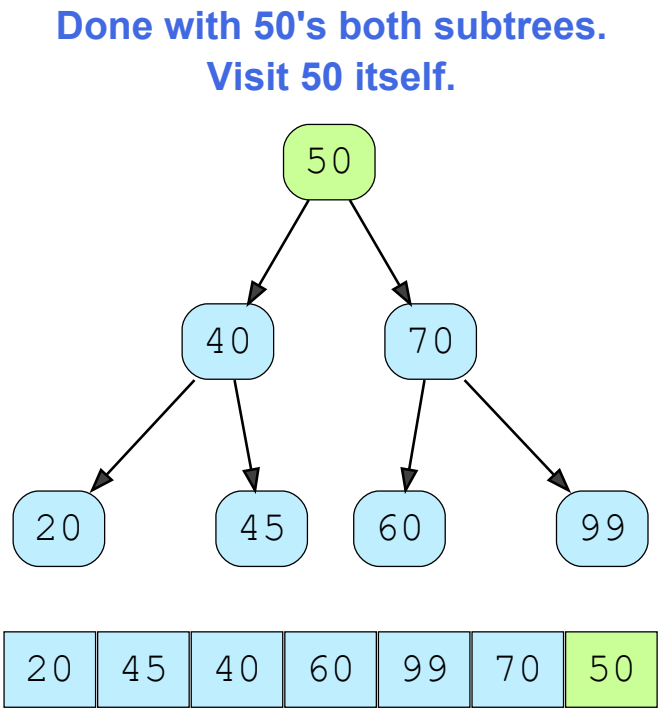


99 is leaf node. Visit it

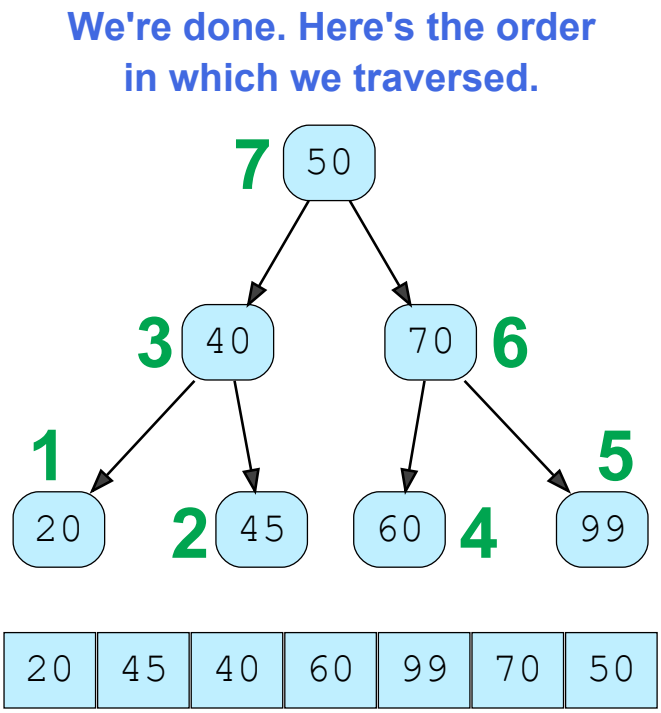


Done with 70's both subtrees.
Visit 70 itself.





14 of 15



15 of 15

Let's take a look at the code for Post Order implementation (which is pretty obvious if you've already looked at the pre-order and in-order traversals).

```
// Returns Keys in the Post Order traversal
BST.prototype.postOrder = function() {
  var output = [];

  function postOrderImpl(node) {
    if (node === null) {
      return;
    }

    // Visit left subtree
    postOrderImpl(node.left);

    // Visit the right subtree
    postOrderImpl(node.right);

    // Visit the node itself.
    output.push(node.data);
  }

  // Call the internal function
  // with Root as the starting point.
  postOrderImpl(this._root);

  return output;
}
```

Traversals in action

Let's look at the traversals in action. We'll insert into a BST and then run all three traversals.

> Run the following code to see traversals in action

JavaScript

HTML

CSS (SCSS)

```
var data = [50, 40, 70, 60, 20, 99, 45];
var bst = new BST();

for (var i = 0; i < data.length; i++) {
  bst.insert(data[i]);
}

console.log("Pre-Order = " + bst.preOrder());
console.log("In-Order = " + bst.inOrder());
console.log("Post-Order = " + bst.postOrder());
```

Console

Clear

```
Pre-Order = 50,40,20,45,70,60,99
In-Order = 20,40,45,50,60,70,99
Post-Order = 20,45,40,60,99,70,50
```

In-Order Predecessor and In-Order Successor

Why do we need to understand In-Order predecessor and In-Order successor?

You might have noticed that we haven't discussed deletion in a BST (we only discussed insertion and search). The reason is that deletion is a little tricky in a BST and understanding In-Order predecessor and successor, helps in implementing deletion from a BST.

For a given node X,

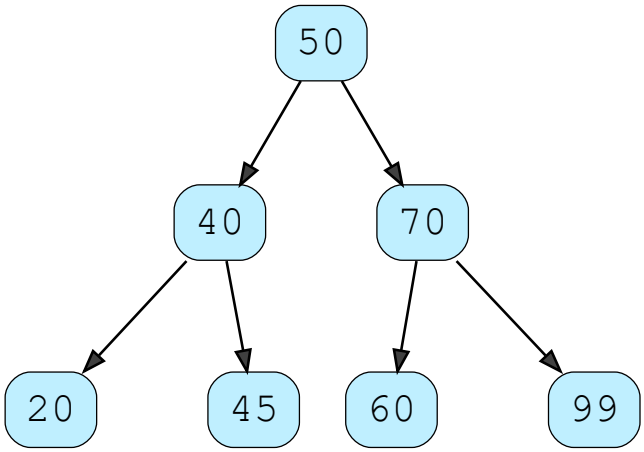
> Node X's ***predecessor*** is the node that comes just before X in InOrder traversal. Also remember that In-Order traversal visits the nodes in a sorted order. Hence, X's predecessor is the node with the largest key smaller than the key of X.

Similarly,

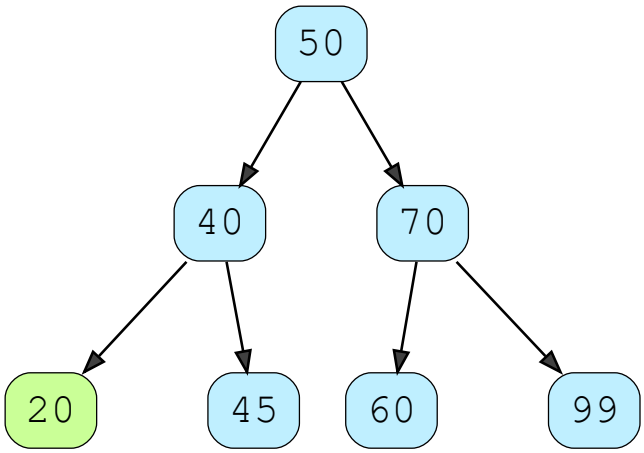
> Node X's ***successor*** is the node that comes right after X in tree's InOrder traversal. As In-Order traversal visits the nodes in a sorted order, it means that X's successor is the node with the smallest key larger than the key of X.

Time for some examples.

Let's look at
Predecessors and Successors

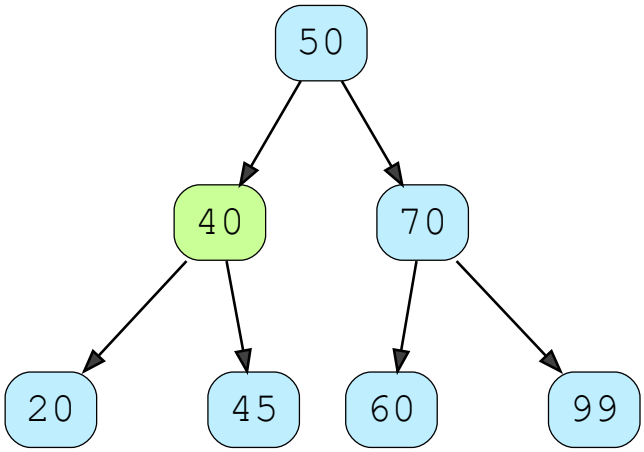


Let's look at 20's
Predecessor and Successor



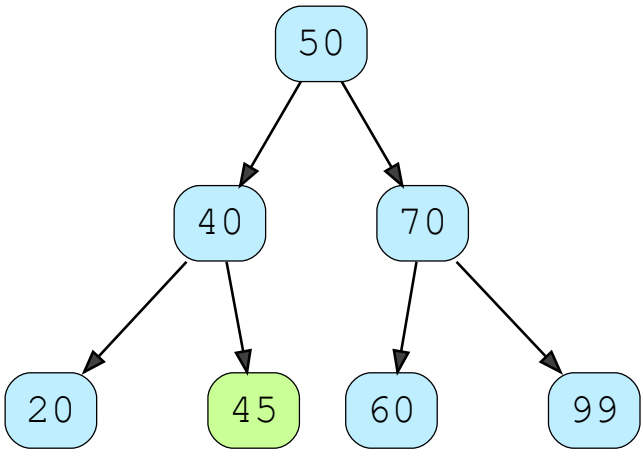
Predecessor is Null ***(no node smaller than 20)***
Successor is 40.

Let's look at 40's Predecessor and Successor

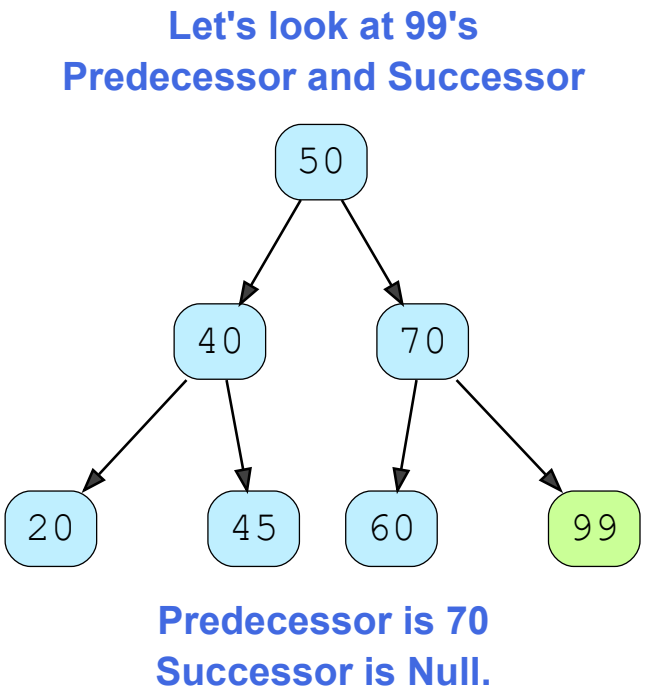


Predecessor is 20
Successor is 45.

Let's look at 45's Predecessor and Successor



Predecessor is 40
Successor is 50.



Deleting a node in a BST

Now that we've understood a lot of terminology about BST's and traversals, let's dive into deleting a Node in a BST. It's a little complicated as we need to ensure that the removal of the node doesn't break the invariant of the BST (left children smaller than the current key and right children larger than the current key).

When deleting a node in BST, there are three cases.

1. It's a leaf node and has no children.
2. The node has one child (either left or right).
3. The node has both left and right children.

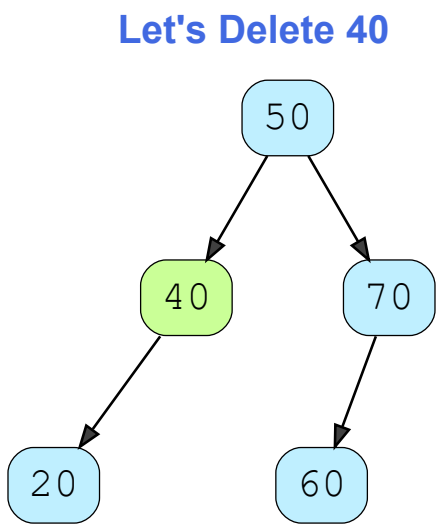
First two cases are easier to handle than the third one. Let's look at the easy ones first.

Deleting a Leaf Node

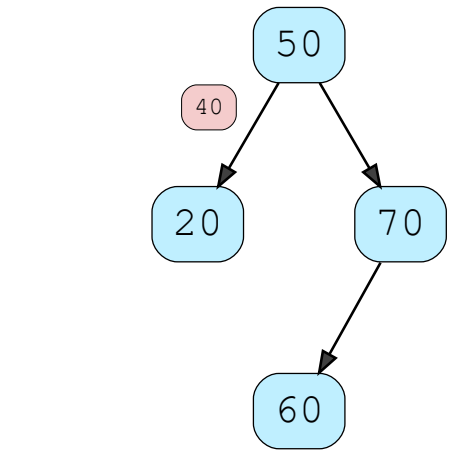
This is the easiest to delete. We just delete it and make sure that it's parent is not pointing to it anymore.

Deleting a node with 1 child

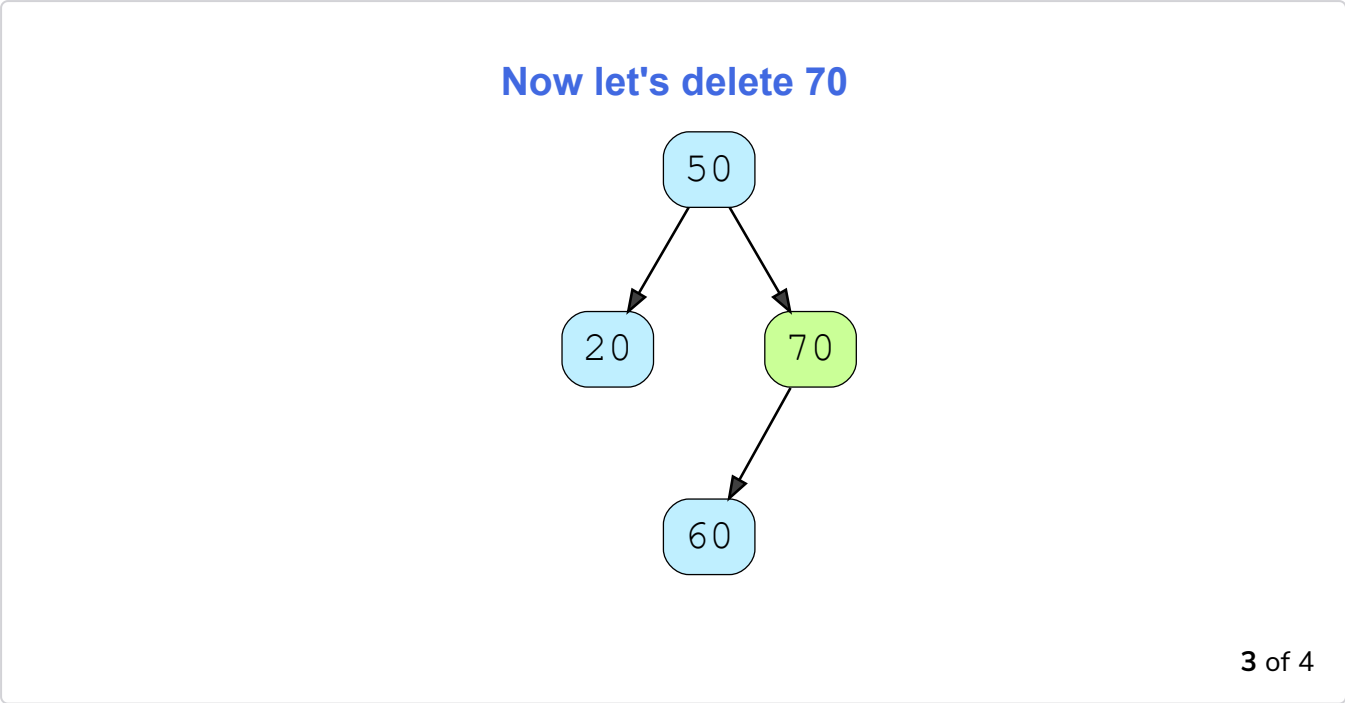
When deleting a node with 1 child, we connect the node's parent with node's child node. In other words, grand parent connects directly with grand child.

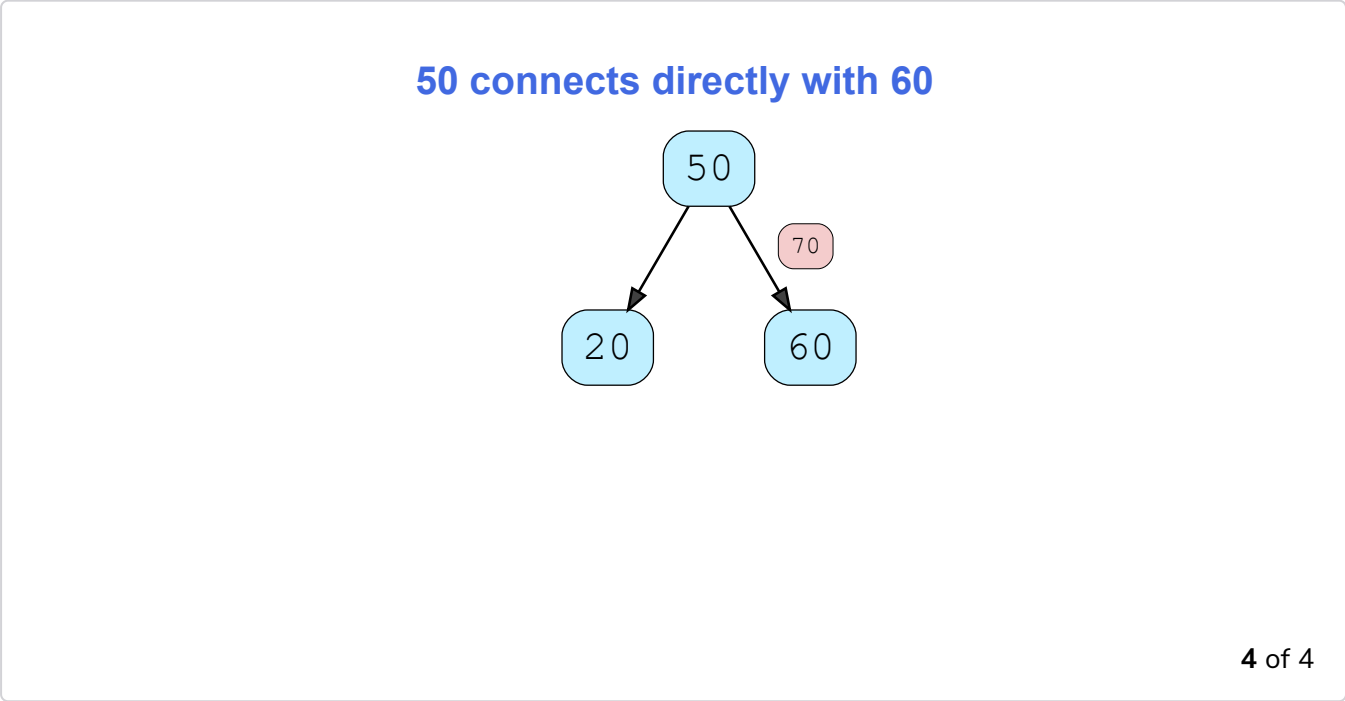


1 of 4



2 of 4





Deleting a node with two children

When we have to delete a node with two children, we have two options. Let's say that the node to be deleted has key X.

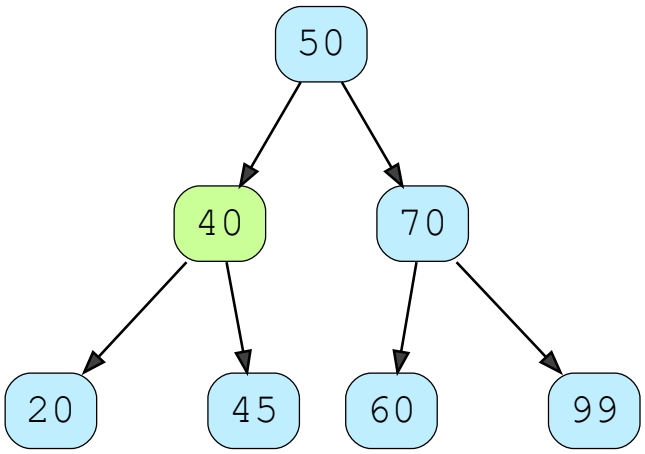
1. Replace the current node's key with its predecessor and then trigger delete for predecessor in node's left subtree.
2. Replace the current node's key with its successor and then trigger delete for successor in node's right subtree.

Why would it work?

Remember predecessor is the highest value smaller than the current key. Now we know that this node has a left subtree. Hence, the predecessor to the current node is somewhere in the left subtree. In fact predecessor is the highest key in the node's left subtree. In addition, as it's the highest node in left subtree, it cannot have a right child. Otherwise that right child would have been the predecessor which ensures that we are now trying to delete a node with zero or one child. We already know that deleting a node with zero or one child is simpler and hence we've reduced the problem from a node with two children to deletion of node with zero or one child.

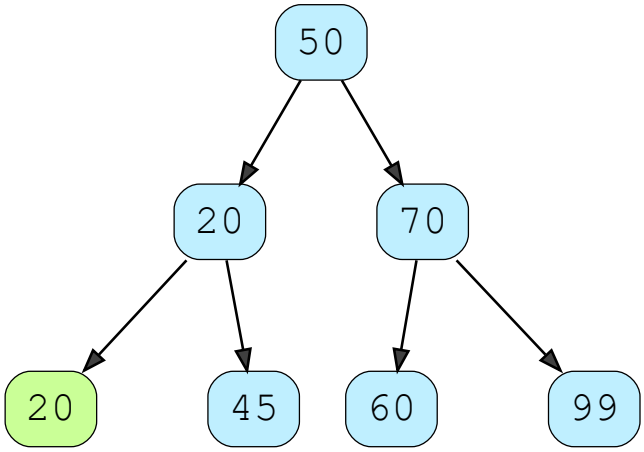
Similarly, successor is the smallest value larger than the current key. We already know that the node has a right subtree. Hence, the successor to the current node is somewhere in the right subtree. In fact, successor is the smallest key in node's right subtree. Hence, it's also a node with zero or one child.

Let's look at a quick visualization

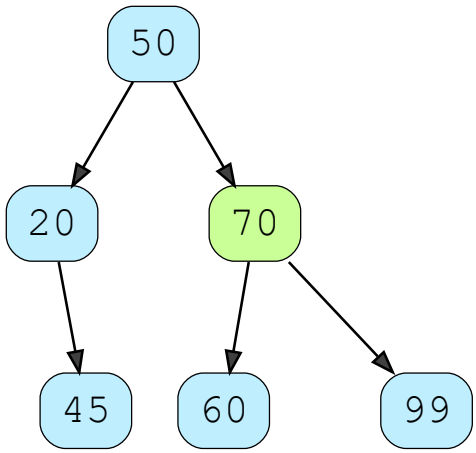


1 of 5

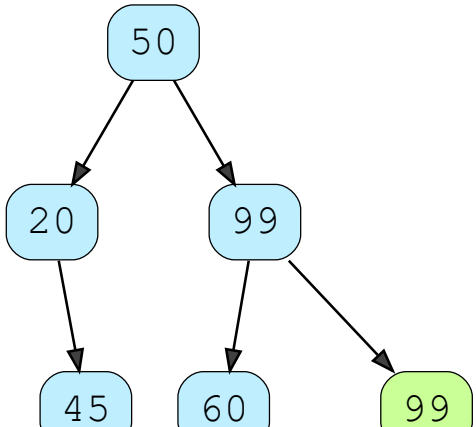
Replace 40 with 20, its predecesssor.
Then delete 20 in node's left subtree.



Now let's delete 70

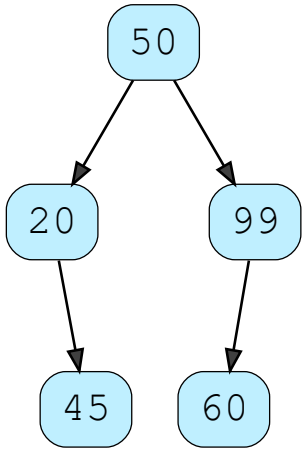


Replace 70 with 99, it's successor.
Then delete 99 in node's right subtree.



4 of 5

Both deletions done.



5 of 5

Implementing Node Removal in BST

We'll implement a remove function that would take a key and delete the key ensuring that BST is left in the correct state after removal of the node. However, before we implement remove function, we need to decide what to do when the node to be deleted has two children. We have the option of either going the route of replacing it with predecessor or with its successor.

Let's pick predecessor. We know that the predecessor in the left subtree of the node will be the highest value of this subtree. Hence, we'll implement a quick maximum function.

```
BST.prototype.maximum = function(node) {
  while (node.right) {
    node = node.right;
  }

  return node.data;
}
```

Now let's implement the actual removal function.

```
BST.prototype.remove = function(key) {
  this.removeImpl(key, this._root);
}

BST.prototype.removeImpl = function removeImpl(key, node) {
  if (node != null) {
    if (key < node.data) {
      // Key might be in the left subtree.
      node.left = this.removeImpl(key, node.left);
    } else if (key > node.data) {
      node.right = this.removeImpl(key, node.right);
    } else {
      // Node found.
      // Let's see if it has two children.
      if (node.left && node.right) {
        // Replace current node with
        // predecessor data
        node.data = this.maximum(node.left);
        node.left = this.removeImpl(node.data, node.left);
      } else {
        // Only 1 child.
        // Let's return the child that's valid.
        node = node.left || node.right;
      }
    }
  }

  return node;
}
```

Let's run the removal code. We'll insert into BST, then delete a few nodes and see that In-Order traversal always returns sorted data.

> Run the following code to try out node removal

JavaScript

HTML

CSS (SCSS)

```
var data = [50, 40, 70, 60, 20, 99, 45];
var bst = new BST();

for (var i = 0; i < data.length; i++) {
  bst.insert(data[i]);
}

console.log("In-Order = " + bst.inOrder());

bst.remove(40);
console.log("In-Order BST after removing 40 = "
            + bst.inOrder());

bst.remove(70);
console.log("In-Order BST after removing 70 = "
            + bst.inOrder());
```

```
+ bst.inorder());
```

Console

Clear

```
In-Order = 20,40,45,50,60,70,99
In-Order BST after removing 40 = 20,45,50,60,70,99
In-Order BST after removing 70 = 20,45,50,60,99
```

Quick Quiz about Trees

1. Trees are what type of data structure

2. Which Traversal returns data in sorted order

Check Answers

Exercise

Exercise

Implement a function to count the number of elements in the tree.

> Some Tests are failing as size method is not implemented. Implement it to fix the test cases.

JavaScript

HTML

CSS (SCSS)

```
BST.prototype.size = function() {
  // Implement this
  return undefined;
}

// This is the method that runs test cases.
// Don't remove this call.
runEvaluation();
```

Console

Clear

```
*** There is some bug lurking there. See failed test cases ***
Here are the tests that ran:
Test case FAILED for bst.size(BST = []. Result: undefined. Expected: 0
Test case FAILED for bst.size(BST = [50]. Result: undefined. Expected: 1
Test case FAILED for bst.size(BST = [40,50]. Result: undefined. Expected: 2
Test case FAILED for bst.size(BST = [40,50,70]. Result: undefined. Expected: 3
```

Summary

- Binary tree is a hierarchical data structure.
- Binary Search Tree (BST) is a binary tree where a node has all smaller keys to its left and keys greater than the node on its right.
- Insertion and Search in a BST are relatively straight forward.

- Removal is a little tricky as deletion of a node can cause the BST to violate its properties.
- Binary Trees can be traversed in many orders. Most common are Pre-Order, In-Order and Post-Order traversals.

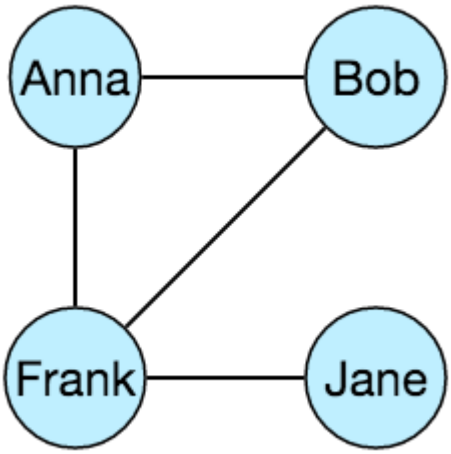
Graphs

This chapter will look at how to work with the graph data structure in JavaScript. We will see at how we can create graphs via Nodes and edges , access elements of a graph via depth wise and breadth wise search.

Introduction

A Graph is a data structure that is helpful in mapping several real world scenarios. e.g. You and your friends on Facebook form a graph called a social graph. Similarly, the roads between towns and cities form a graph commonly know as 'Map'.

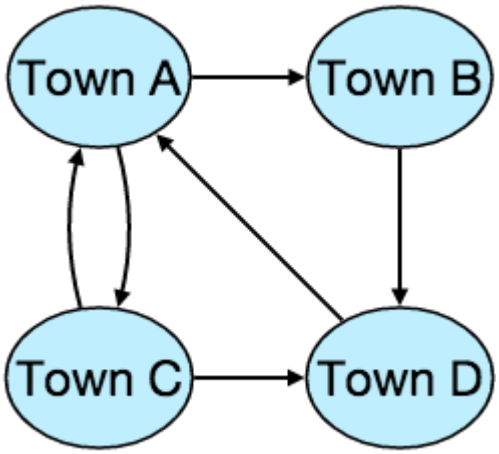
In computer science, a graph consists of a set of vertices and a set of edges. The vertices are the entities and edges define relationships between those entities. e.g. Bob, Frank and Anna are friends. Frank has a friend named Jane who isn't friend with Bob and Anna. In this example, all four people are vertices and edges represent a direct friendship. Let's look at the following graph



Friendship Graph

The above graph shows edges that don't have any direction. Such graphs are called ***Undirected Graphs***. Undirected graphs show that the connection goes both ways. Here we are using an undirected graph as friendship is a mutual relationship (or at least it is supposed to be).

The edges with arrows represent direction of the relationship and such graphs are called ***Directed Graphs***. e.g. let's draw a map of places where a directed edge represents that a road connects vertex A to B. If the edge goes from A to B, it means that the there is a road (one-way road) from A to B. Let's see an example



Graph of Roads

The above graph shows that if you are in Town A, you cannot directly go to Town D. You would either have to go through Town B or Town D. Similarly, from Town D, you cannot go to anywhere except Town A etc.

We've discussed some theory about graphs. Let's see how can we represent vertices and edges in a graph.

Representing Graph in Javascript

We've already established that a graph is a set of vertices and a set of edges. Representing vertices is easy. It's just a list of vertex names. Edges is a little tricky. For every edge, we need to store a source and destination vertex. There are two common ways to store edges. They are called

1. Adjacency Matrix
2. Adjacency List

Let's look at both of these techniques.

Adjacency Matrix

Adjacency Matrix is a square matrix (table) where first row and first column consists of vertices in the graph. The rest of the cells contain either 0 or 1 (it could be another number for weighted graphs but let's ignore that for a while). So each Row x Column intersection points to a cell and the value of that cell dictates whether the vertex represented by Row and vertex represented by Column are connected or not. If the value is 1 for cell V1 x V2, then V1 is connected to V2. An example would help here. Let's create the adjacency matrix for the town graph we saw above.

	Town A	Town B	Town C	Town D
Town A	0	1	1	0
Town B	0	0	0	1
Town C	1	0	0	1
Town D	1	0	0	0

Adjacency Matrix for our City

Now if you have to tell whether there is a direct road between TownA and Town D, you just look at TownA x TownD cell which is 0 so there is no road between Down D.

When we have an undirected graph, we call it a ***symmetric adjacency matrix*** because if a graph is undirected then, if V1 x V2 is an edge, then V2 x V1 will also be an edge. To clarify, let's look at the adjacency matrix for the friend graph that we saw above.

	Anna	Bob	Frank	Jane
Anna	0	1	1	0
Bob	1	0	1	0
Frank	1	1	0	1
Jane	0	0	1	0

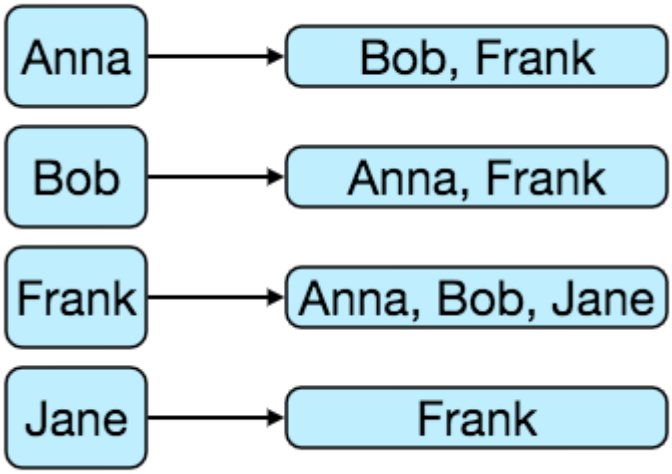
Adjacency Matrix of friendship Graph

Adjacency List

If you notice, in above adjacency matrices, there are a lot of zeroes. As graphs grow, graphs might have far less number of connections than the vertices. Hence, storing non-existence of an edge becomes wasteful e.g. Facebook has 1.5 Billion users but each user has 200-300 people as friends on average. If Facebook stored this friendship graph as an adjacency matrix, it will be a matrix of quintillion rows and columns. It's almost impossible to store such a big matrix. However, the key thing to notice here is that it will be a very sparse matrix and most of the cells will be zero. It makes sense to only store the connections that exist and if the connection doesn't exist, we assume that it's non-existent. This is called an adjacency list.

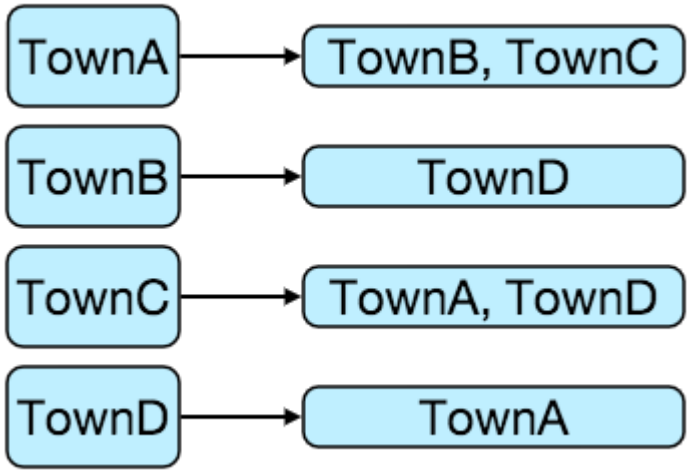
More precisely, Adjacency list is an array of lists where each element is the array is a vertex and each vertex points to a list of vertices it connects to.

Let's look our friendship graph stored as an adjacency list.



Adjacency list for Friendship Graph

Similarly, here's the adjacency list for our city graph.



Adjacency list for Friendship Graph

In our implementation, we are going to opt for Adjacency Lists over Adjacency Matrix as adjacency lists are more space efficient. However, if you end up in a scenario where your adjacency matrix is going to be nearly full, you should opt for adjacency matrix.

Let's have a quick quiz on graphs

Graphs

1 When the edges of a graph have a direction , that type of graph is known as

2 A vertex in the Graph is also referred to as

Check Answers

Implementing a Graph in Javascript

For simplicity, we assume that the vertices are either strings or integers and a string or its equivalent integer points to the same node (If you have followed the Dictionary tutorial, you probably know where are we going with this. We are going to use a Javascript object to store list of vertices just like we did in the dictionary). Alternatively, we could have used our Dictionary data structure here. Here's our Graph data structure.

```
function Graph() {
  this._vertices = {};
  this._vertexCount = 0;
}
```

Adding a Vertex to Graph

For adding a vertex, we add the vertex to our vertices object. The key is the name of the vertex and value is another object which is actually going to be the list of edges.

```
Graph.prototype.addVertex = function(v) {
  var vertexType = typeof(v);

  if (vertexType != 'number' &&
      vertexType != 'string') {
    throw 'Vertex can only be a string or number';
  }

  if (this._vertices.hasOwnProperty(v)) {
    throw 'Duplicate Vertex is not allowed';
  }

  this._vertices[v] = {};
  this._vertexCount++;
}
```

Adding Edges to a vertex

Adding edges to a vertex is easy. We'll just add the edges to the vertex's adjacency list. Only thing to notice here is that we ensure that the edges exist as vertices in our graph.

```
Graph.prototype.addEdges = function(v, edges) {
  if (!this._vertices.hasOwnProperty(v)) {
    throw 'Vertex not found';
  }

  var edgesObj = this._vertices[v];

  for (var i = 0; i < edges.length; i++) {
    var edge = edges[i];

    if (this._vertices.hasOwnProperty(v)) {
      throw 'Invalid vertex cannot be added as edge';
        }

    edgesObj[edge] = true;
  }
}
```

Getters for Vertices and Edges

Implementing Getters is quite straightforward. We will iterate over the lists (both vertices and edges) and return them. Let's look at how are we implementing getter for vertices.

```
Graph.prototype.getVertices = function() {
  var vertices = [];
  for (var v in this._vertices) {
    vertices.push(v);
  }

  return vertices;
}
```

... and similarly, getter for edges given a vertex is implemented below.

```
Graph.prototype.getEdges = function(v) {
  if (!this._vertices.hasOwnProperty(v)) {
    throw 'Vertex not found';
  }

  var edgesObj = this._vertices[v];
  var edges = [];
```

```
  for (var e in edgesObj) {
    edges.push(e);
  }

  return edges;
}
```

Let's see our Graph code in action

It's time to see our graph code in action. We'll create the friendship graph that we've been using as example.

> Run the following code to see Graph in action.

JavaScript

HTML

CSS (SCSS)

```
var graph = new Graph();
graph.addVertex('Anna');
graph.addVertex('Bob');
graph.addVertex('Frank');
graph.addVertex('Jane');

graph.addEdges('Anna', ['Bob', 'Frank']);
graph.addEdges('Bob', ['Anna', 'Frank']);
graph.addEdges('Frank', ['Anna', 'Bob', 'Jane']);
graph.addEdges('Jane', ['Frank']);

var vertices = graph.getVertices();
for(var i=0; i<vertices.length; i++){
  var edges = graph.getEdges(vertices[i]);
  console.log(vertices[i] + ' is friends with '+ edges);
}
```

Console

Clear

```
Anna is friends with Bob,Frank
Bob is friends with Anna,Frank
Frank is friends with Anna,Bob,Jane
Jane is friends with Frank
```

Graph Search (or Traversal) Algorithms

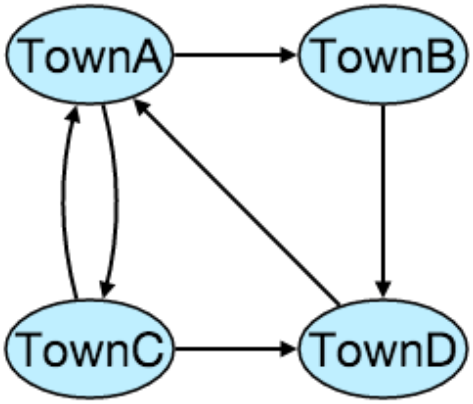
There are two most common ways to traverse a graph given a start node. There are many other ways to traverse a graph depending on the nature of the graph (e.g. Tree is a special graph and we've seen that we use Pre-Order, In-Order and Post-Order traversals for the tree). Here are the most common Graph traversal algorithms.

Depth First Search

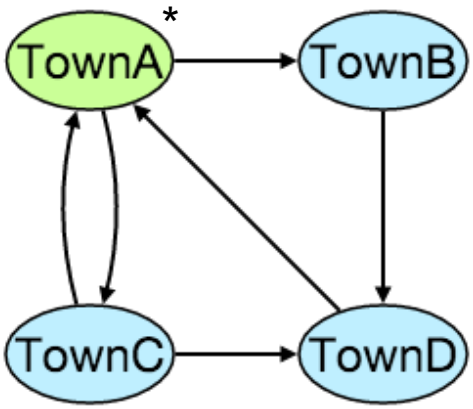
Breadth First Search

Depth First search in the graph

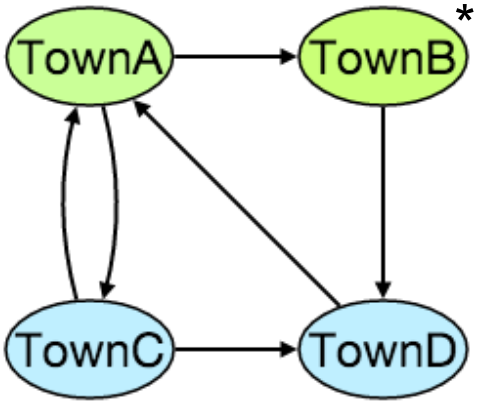
Depth First Search (DFS) is a traversal strategy in which we start from a node and then traverse the graph as far as possible before backtracking and starting traversal from the next node. At each step, we mark the nodes that we've already visited so that we don't visit them again. Here's a visualization of DFS starting at "Town A" in our city graph.



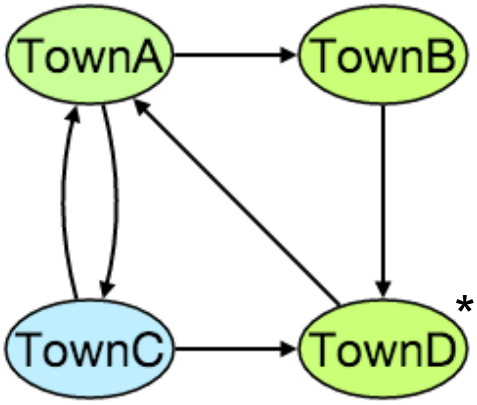
DFS - start at Town A



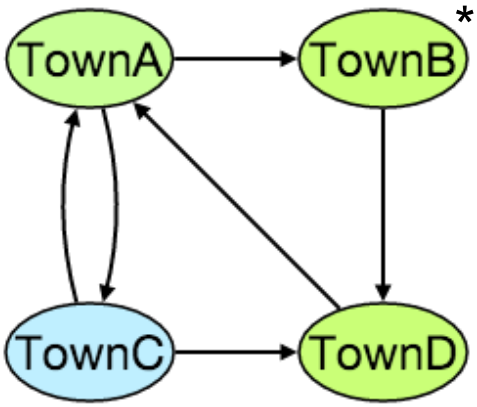
TownA is marked as visited.
Visit TownB



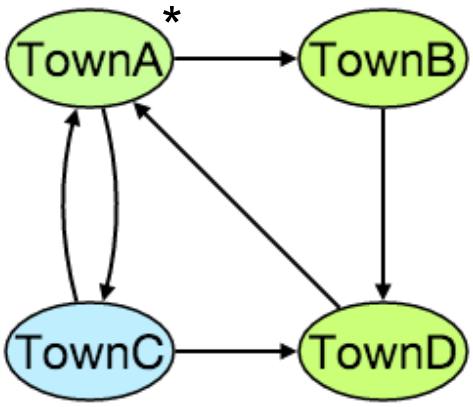
TownB is marked as visited.
Visit TownD.



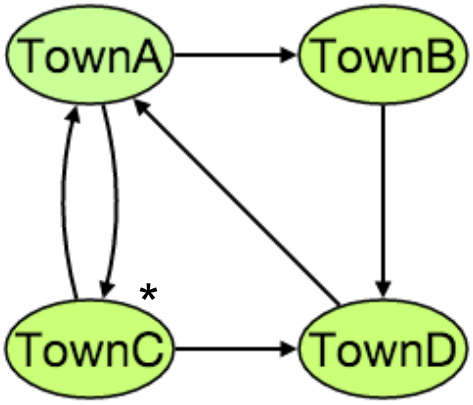
TownD is marked as visited.
Visit TownA but A is already visited.



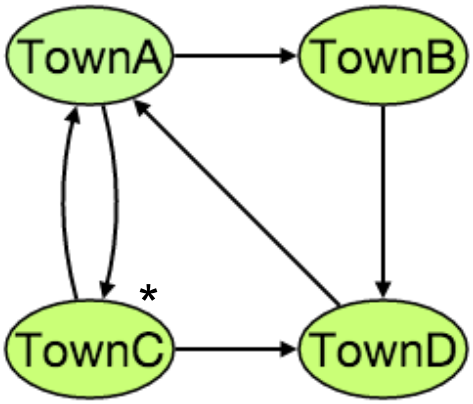
We backtrack to B. B has no more edges so we backtrack to A.



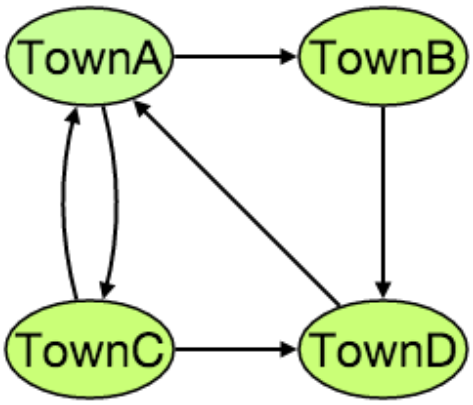
We are at A and we figure that it has another edge so Visit TownC



TownC is marked visited.



We now try going to A from C but A is already visited.



Similarly, all other nodes are already visited.So we are done.

Here's the code for Depth First Search

```
var DFS = function(graph) {
  var vertices = graph.getVertices();
  if (vertices.length === 0) {
    return;
  }

  // Mark all vertices as NOT VISITED at start
  var visited = {};
  for (var i = 0; i < vertices.length; i++) {
    visited[vertices[i]] = false;
  }
```

```
  }

  // Define our DFS impl method
  function DFSImpl(v) {
    visited[v] = true;
    console.log('Visiting Vertex: ' + v);

    var edges = graph.getEdges(v);
    for (var j = 0; j < edges.length; j++) {
      var edge = edges[j];
      if (!visited[edge]) {
       DFSImpl(edge);
      }
    }
  }

  // Start DFS
  for (var i = 0; i < vertices.length; i++) {
    var vertex = vertices[i];
    if (!visited[vertex]) {
      DFSImpl(vertex);
    }
  }
};
```

> *Run the following code to see DFS in action*

JavaScript

HTML

CSS (SCSS)

```
var graph = new Graph();
graph.addVertex('Anna');
graph.addVertex('Bob');
graph.addVertex('Frank');
graph.addVertex('Jane');

graph.addEdges('Anna', ['Bob', 'Frank']);
graph.addEdges('Bob', ['Anna', 'Frank']);
graph.addEdges('Frank', ['Anna', 'Bob', 'Jane']);
graph.addEdges('Jane', ['Frank']);

DFS(graph);
```

Console

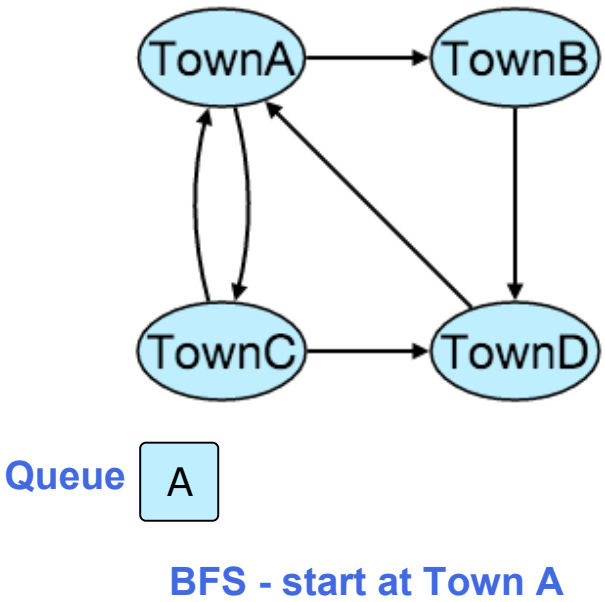
Clear

```
Visiting Vertex: Anna
Visiting Vertex: Bob
```

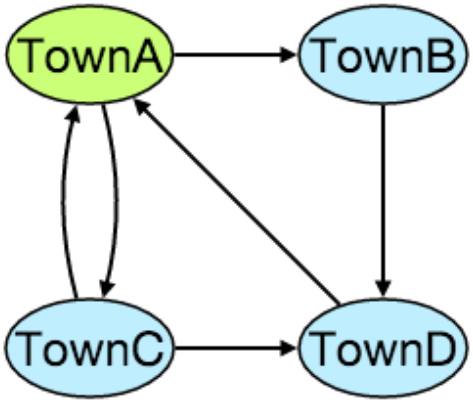
```
Visiting Vertex: Frank
```

Breadth First search

A breadth-first search (BFS) starts at a first vertex and tries to visit vertices as close to the first vertex as possible. In essence, this search moves through a graph layer by layer, first examining layers closer to the first vertex and then moving down to the layers farthest away from the starting vertex. We achieve this by maintaining a queue of the edges connected to the node that we are visiting. We then pickup the next vertex to visit from the queue. Here's a quick visualization

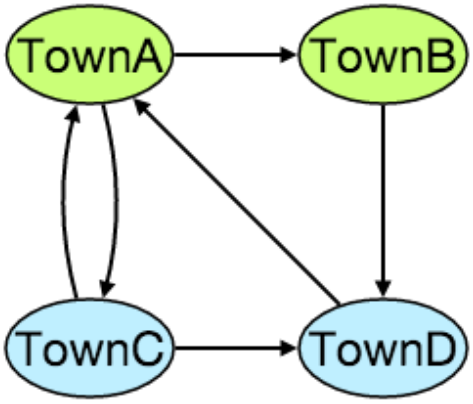


1 of 6



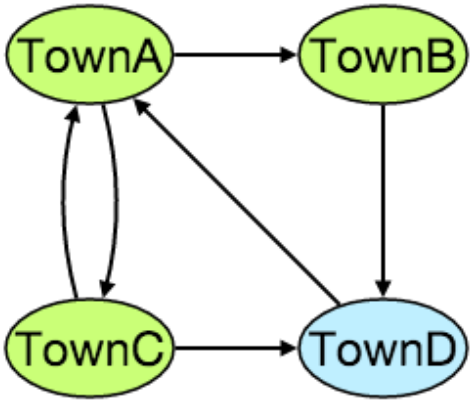
Queue B C

Visit A. Put B, C in Queue.



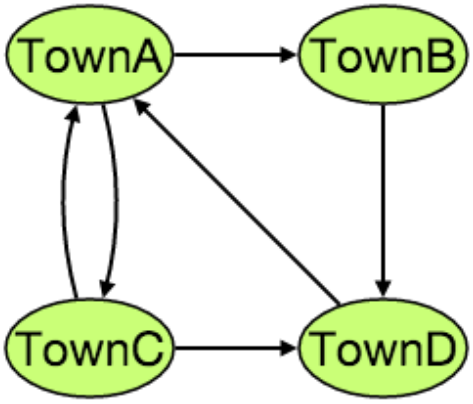
Queue C D

Visit B. Put D in Queue.



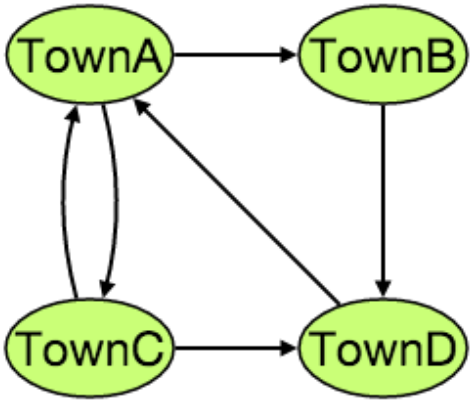
Queue C D

Visit C. Don't put A or D in queue as they are already visisted.



Queue D

Visit D from the queue.



Queue *<empty>*

Queue is empty so we are done.

Here's the code for BFS.

```
var BFS = function(graph) {
  var vertices = graph.getVertices();
  if (vertices.length === 0) {
    return;
  }

  // Mark all vertices as NOT VISITED at start
  var visited = {};
  for (var i = 0; i < vertices.length; i++) {
    visited[vertices[i]] = false;
  }

  var queue = [];
  var startV = vertices[0];
  queue.push(startV);
  visited[startV] = true;

  while (queue.length > 0) {
    // Inefficient. use the Wueue that
    // we created in Queue tutorial
    var v = queue.shift();
    console.log('Visited: ' + v);
    var edges = graph.getEdges(v);

    for (var i = 0; i < edges.length; i++) {
      var e = edges[i];
      if (!visited[e]) {
        queue.push(e);
```

```
        queue.push(e);
        visited[e] = true;
      }
    }
  }
};
```

> Run following code to see BFS in action

JavaScript

HTML

CSS (SCSS)

```
var graph = new Graph();
graph.addVertex('Anna');
graph.addVertex('Bob');
graph.addVertex('Frank');
graph.addVertex('Jane');

graph.addEdges('Anna', ['Bob', 'Frank']);
graph.addEdges('Bob', ['Anna', 'Frank']);
graph.addEdges('Frank', ['Anna', 'Bob', 'Jane']);
graph.addEdges('Jane', ['Frank']);

BFS(graph);
```

Console

Clear

```
Visited: Anna
Visited: Bob
Visited: Frank
Visited: Jane
```

Exercise 1

In our first exercise, we will implement a function to count the total number of edges in a graph.

> Some Tests are failing as the countTotalEdges method is not implemented. Implement it to fix the test cases.

JavaScript

HTML

CSS (SCSS)

```
Graph.prototype.countTotalEdges = function() {
  // Implement this
  return undefined;
}

// This is the method that runs test cases.
// Don't remove this call.
runEvaluation();
```

Console

Clear

```
*** There is some bug lurking there. See failed test cases ***
Here are the tests that ran:
Test case FAILED for Empty graph edges count. Result: undefined. Expected: 0
Test case FAILED for Graph with just one vertex. Result: undefined. Expected: 0
Test case FAILED for Graph with two edges. Result: undefined. Expected: 2
Test case FAILED for Graph with 8 edges. Result: undefined. Expected: 8
```

Summary

A graph is a set of vertices and edges.

Graphs can be represented using either adjacency matrices or adjacency list.

Lookups are fast in adjacency matrices but adjacency lists take up less space in memory.

Two most common algorithms to traverse the graph are called Depth First Search (DFS) and Breadth First Search (BFS).